

1. Introducción y Contextualización

1.1. Introducción

Este Trabajo de Fin de Grado se lleva a cabo con la empresa Eseleos SL, la cual trabaja en distintos proyectos de software, incluyendo uno llamado Revivack. Revivack es un proyecto que desarrolla un sistema de recuperación de objetos para abordar el desafío de los residuos generados por productos cotidianos. Trabajando con distintas empresas, Revivack logra trazar productos de donantes gracias a la tecnología Blockchain (Pedragosa 2023).

La aplicación web desarrollada en este trabajo es un método mediante el cual diversos usuarios pueden devolver distintos productos a sus respectivas empresas, iniciando así una cadena que cierra el ciclo de vida de estos productos.

1.2. Definición de Conceptos

- **Usuario:** Un usuario es todo individuo o entidad que interactúa con el sistema, puede ser tanto un donante como una empresa.
- **Empresa:** Una empresa es un tipo de usuario que puede ofrecer productos con sus respectivas rutas para que los donantes los entreguen, comenzando así a crear una cadena.
- **Donante:** Un donante es un tipo de usuario que contribuye con productos a una empresa. Los donantes facilitan la entrega de productos a través de rutas específicas. Este usuario puede llegar a recibir recompensas por parte de las empresas al retornar distintos productos.
- **Producto:** Un producto es cualquier bien o artículo ofrecido por una empresa que puede ser donado por un donante. Estos productos son proporcionados con rutas específicas para facilitar su entrega y distribución.
- **Ruta** Una ruta es el medio específico definido por una empresa para el retorno de un producto desde una región particular a uno de los puntos de recogida. Por ejemplo, los retornos desde la península al almacén de Madrid se realizan a través de Correos.
- **Cadena:** Una cadena es el proceso que sigue un producto desde su retorno y descomposición hasta la creación de un nuevo producto utilizando sus materiales, empleando la

tecnología Blockchain para asegurar la transparencia y trazabilidad en cada etapa de su ciclo de vida.

1.3. Contextualización General

Hoy en día, el ser humano, dotado de tecnología avanzada y conocimiento científico, enfrenta el desafío de manejar la contaminación derivada de objetos más complejos que no tienen alternativas de reciclaje tan evidentes. A pesar de la creciente conciencia sobre la importancia del reciclaje, nos encontramos con productos como aparatos electrónicos, baterías, neumáticos y otros artículos que contienen materiales difíciles de descomponer y reciclar adecuadamente. Esta falta de alternativas efectivas para reciclar estos objetos conduce a su acumulación en vertederos o a su eliminación inadecuada, lo que resulta en la contaminación del suelo, el agua y el aire. Es crucial que avancemos en la investigación y el desarrollo de tecnologías de reciclaje más avanzadas y eficientes para abordar este problema y reducir nuestra dependencia de los recursos naturales finitos. Además, es necesario fomentar la responsabilidad del productor para que se haga cargo del ciclo de vida completo de sus productos, desde la fabricación hasta la disposición final, promoviendo así una economía circular que minimice el impacto ambiental y preserve los recursos para las generaciones futuras.

Además del desafío de reciclar objetos complejos, enfrentamos el problema del greenwashing, donde empresas promueven una imagen de responsabilidad ambiental sin acciones reales. Esto confunde a los consumidores y obstaculiza la verdadera sostenibilidad. Es crucial que los consumidores sean críticos y que las regulaciones sean más estrictas para combatir esta práctica y promover una verdadera responsabilidad corporativa. En este contexto, la economía circular emerge como una solución clave. Promueve el diseño de productos con la capacidad de ser reutilizados y reciclados, reduciendo así la generación de desechos y promoviendo un uso más eficiente de los recursos. La implementación de modelos de negocio basados en la economía circular puede ayudar a cerrar el ciclo de vida de los productos y a minimizar su impacto ambiental, promoviendo al mismo tiempo la innovación y la creación de empleo en sectores más sostenibles.

1.4. Stakeholders

Los stakeholders son todas aquellas partes que tienen algún tipo de interés y relación tanto interna como externa con el proyecto. A continuación se exponen los distintos stakeholders del proyecto:

- **Ponente del TFG:** Carles Farré Tost, del 'Departament d'Enginyeria de Serveis i Sistemes d'Informació' (ESSI), el cual supervisará el proyecto para su correcta realización.
- **Eseleos SL:** La empresa que se encarga de proveer desarrolladores a distintos proyectos, incluyendo el de Revivack.
- **Desarrolladores y Diseñadores:** Son las personas encargadas de diseñar e implementar el software y las distintas pantallas de la WebApp.
- **Clientes:** Tanto las empresas implicadas en el proyecto como los usuarios potenciales son los que van a utilizar la aplicación cuando esta se finalice.

6. Especificación

6.1. Requisitos

A continuación, se introducen los distintos requisitos funcionales y no funcionales (empleando la clasificación Volere⁵ para estos últimos) con los que cuenta el sistema:

6.1.1. Requisitos Funcionales

- Un usuario puede ser tanto un donante como una empresa.
- El sistema ha de estar disponible tanto en castellano como en inglés.
- El sistema ha de permitir registrar usuarios nuevos al sistema.
- El sistema ha de poder autenticar a usuarios existentes a través del email.
- El sistema ha de permitir cerrar sesión a los usuarios.
- El sistema ha de permitir a usuarios logueados o no ver los perfiles de las distintas empresas así como sus productos.
- El sistema ha de poder guardar todas las imágenes de todos los productos/usuarios del sistema.
- Una empresa ha de poder modificar su perfil/página pública en todo momento, al igual que previsualizarlo mientras está editando.
- Una empresa ha de tener acceso al dashboard del sistema para poder utilizar las distintas funcionalidades internas e información adicional (estadísticas, valores, etc.).
- Una empresa ha de poder dar de alta un producto y que éste sea visible en su perfil/página pública.
- Una empresa ha de poder dar de alta una ruta para vincularla a sus productos.
- Un usuario ha de poder devolver un producto dado de alta en una empresa mediante el uso de sus rutas.

⁵La plantilla de especificación de requisitos Volere está creada para ser utilizada como una base para las especificaciones de requisitos (Robertson 2006).

- Todo producto ha de tener una ruta predeterminada en la que el usuario lo devuelve por su cuenta y con sus propios medios.
- Toda ruta ha de tener un presupuesto asociado (puede ser 0) el cual el usuario ha de poder pagar mediante el uso del sistema a la hora de retornar el producto.
- El sistema ha de contar con un sistema de afiliación/planes para empresas que quieran tener ciertas ventajas (cuenta con tres planes: gratuito, básico y premium).

6.1.2. Requisitos No Funcionales

Nombre	11b. Requisitos de Personalización e Internacionalización
Descripción	El sistema ha de estar disponible en distintos idiomas
Justificación	Para llegar a más usuarios y que su experiencia sea la mejor posible
Criterio de Satisfacción	El sistema estará tanto en español como en inglés (verificados por un traductor).

Volere	11a. Requisitos de Facilidad de Utilización
Descripción	La interfaz de usuario debe ser intuitiva y fácil de usar para usuarios no técnicos.
Justificación	Para mejorar la adopción y satisfacción del usuario.
Criterio de Satisfacción	Realización de pruebas de usabilidad con al menos 10 usuarios representativos, logrando una puntuación de satisfacción mínima de 8/10.

Volere	15c. Requisitos de Privacidad
Descripción	El sistema debe proteger la información de los usuarios mediante la autenticación segura.
Justificación	Para proteger los datos sensibles de los usuarios y cumplir con las regulaciones de protección de datos.
Criterio de Satisfacción	El sistema debe pasar auditorías de seguridad sin vulnerabilidades críticas.

Volere	12e. Requisitos de Fortaleza o Tolerancia a Fallas
Descripción	El sistema debe tener un tiempo de actividad del 99.9 % y ser capaz de recuperarse de fallos en menos de 1 hora.
Justificación	Para garantizar la disponibilidad continua del servicio.
Criterio de Satisfacción	Monitoreo de tiempo de actividad que demuestre un uptime del 99.9 % y pruebas de recuperación de desastres que cumplan con el tiempo de recuperación especificado.

Volere	14a. Requisitos de Mantenimiento
Descripción	El sistema debe ser fácil de mantener y actualizar con una documentación clara y completa.
Justificación	Para reducir el costo y el tiempo de mantenimiento.
Criterio de Satisfacción	Documentación completa y actualizada, y tiempo promedio de implementación de actualizaciones y resolución de problemas no superior a 2 horas.

Volere	14c. Requisitos de Adaptabilidad
Descripción	El sistema debe ser compatible con los navegadores web más comunes (Chrome, Firefox, Safari, Edge) y dispositivos móviles.
Justificación	Para asegurar que todos los usuarios puedan acceder al sistema independientemente del dispositivo o navegador que utilicen.
Criterio de Satisfacción	Pruebas de compatibilidad que demuestren funcionalidad en los principales navegadores y dispositivos móviles.

Volere	12g. Requisitos de Expansión o Crecimiento
Descripción	El sistema debe ser escalable para manejar un aumento en el número de usuarios y volumen de datos.
Justificación	Para adaptarse al crecimiento futuro y asegurar el rendimiento.
Criterio de Satisfacción	Capacidad para aumentar los recursos del servidor y mantener el rendimiento con un incremento del 50 % en la carga de trabajo.

Volere	15a Requisito de Acceso
Descripción	Los productos/rutas/perfil de una empresa solo pueden ser editables y accesibles por esa empresa.
Justificación	Para proteger la integridad y confidencialidad de los datos de la empresa.
Criterio de Satisfacción	Comprobar que solo los usuario empresa puede acceder a sus productos/rutas/perfil.

6.2. Casos de Uso

Los casos de uso sirven para describir cómo interactúan los usuarios (o actores) con un sistema para lograr un objetivo específico (Vega 2010). A continuación se presentan tres diagramas de casos de uso para distintos tipos de usuarios:

Todo tipo de usuarios (Donante/Empresa)

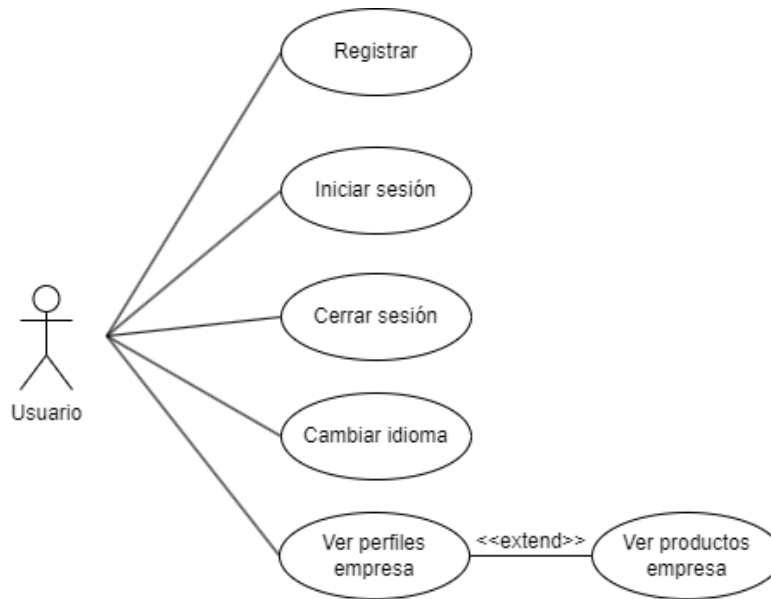


Figura 2: Diagrama de casos de uso para todo tipo de usuarios. Fuente: elaboración propia.

En la figura 2 se puede observar el diagrama de casos de uso de cualquier usuario del sistema, es decir, tanto usuarios donante como usuarios empresa. Este tipo de usuarios pueden realizar las siguientes funcionalidades:

- **Registrarse:** Todo usuario de la aplicación tiene la opción de crear una cuenta registrándose en el sistema como empresa o donante.
- **Iniciar sesión:** Todo usuario registrado puede iniciar sesión en el sistema con su email a través de un MagicLink⁶.
- **Cerrar sesión:** Todo usuario logueado tiene la opción de cerrar sesión en cualquier momento.
- **Cambiar de idioma:** todo usuario logueado tiene la posibilidad de cambiar de idioma entre español e inglés.

⁶El magic link es un correo que se envía al email indicado por el usuario en el caso de estar registrado para poder acceder al sistema a través de él.

- **Ver perfiles empresa:** Todo usuario de la aplicación (logueado o no) puede ver los perfiles de las distintas empresas dadas de altas y con el perfil activo en el sistema.
- **Ver productos empresa:** Todo usuario de la aplicación (logueado o no) puede ver los productos publicados en los perfiles de las distintas empresas del sistema.

Usuarios Donante

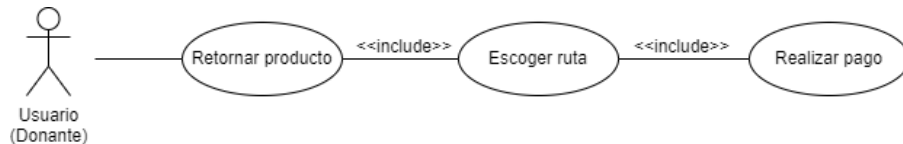


Figura 3: Diagrama de casos uso para usuarios donante. Fuente: elaboración propia.

En la figura 3 se puede ver el diagrama de casos de uso de usuarios donante. Este tipo de usuarios pueden realizar las siguientes funcionalidades:

- **Retornar producto:** Todo usuario donante puede retornar cualquier producto disponible para retornar en los perfiles de las distintas empresas del sistema, al retornar un producto, el usuario ha de escoger la ruta y realizar el pago correspondiente al retorno del producto (el cual puede ser 0 y no pagar nada).

Usuarios Empresa



Figura 4: Diagrama de casos uso para usuarios donante. Fuente: elaboración propia.

En la figura 4 se puede ver el diagrama de casos de uso de los usuarios registrados como empresa en el sistema:

- **Acceso al dashboard:** Todo usuario empresa tiene acceso al dashboard principal del sistema con información general de su perfil y acceso a las distintas funcionalidades del sistema (siguientes casos de uso de empresa). La información que posee es la siguiente:
 - El porcentaje de completado del perfil público de la empresa, así como si esta está certificada o no (Una empresa está certificada cuando tiene al menos un producto con una ruta activa).
 - La cantidad de productos que tiene dados de alta en el sistema.
 - La cantidad de rutas creadas y el número total de productos recibidos.

- El número de cadenas comenzadas y la cantidad de residuos evitados (Estas funciones están deshabilitadas).
- **Editar perfil:** Todo usuario empresa puede editar su perfil público en cualquier momento, pudiendo editar los siguientes campos:
 - **La URL del perfil público:** se puede editar la dirección URL del perfil público de la empresa mediante la cual otros usuarios pueden acceder a él.
 - **Dirección web de la entidad:** se pueden añadir hasta cuatro direcciones web de interés externas al sistema.
 - **Localizaciones:** Se pueden añadir hasta seis localizaciones de interés de la empresa (dos por cada grupo: Oficina central, Sucursal o delegación y Centro de recuperación).
 - **Logo e Imagen corporativa:** Se puede cambiar tanto el logo como la imagen corporativa que aparecen en el perfil público.
 - **Descripción:** Se puede editar la descripción de la empresa.
 - **Información sobre sostenibilidad:** Se puede añadir información extra sobre cómo la entidad ayuda al medio ambiente y contribuye a la sostenibilidad.
 - **Publicar:** La empresa puede decidir si publicar o no el perfil para que sea accesible para el resto de usuarios.
 - **Gestión de productos:** Todo usuario empresa tiene acceso a una tabla con todos los productos creados, desde esta tabla puede:
 - **Crear un nuevo producto:** el usuario empresa puede crear nuevos productos, incluyendo la siguiente información sobre estos:
 - Nombre del producto.
 - A que categoría y subcategoría pertenece este producto.
 - Peso aproximado del producto en kilogramos.
 - Dimensiones aproximadas del producto en centímetros (ancho, largo y alto).
 - Hasta cuatro imágenes del producto (con un mínimo de una imagen y un tamaño máximo de 5MB).
 - Descripción del producto.

- Indicación de si el producto dispone de servicio de reparación y, de ser así, que servicio.
- Indicación de si se ofrece alguna recompensa a usuarios donantes que retornen el producto y, de ser así, que recompensa.
- Indicación de si también reciben el mismo producto de otras marcas y, de ser así, de que marcas.
- Indicación de como se recibe el producto (Si mediante las rutas del sistema, a cuenta de la empresa o ambas opción).
- Indicación de si el producto está publicado o no (visible para el resto de usuarios o no).

- Después de crear el producto se le pueden asociar rutas ya dadas de alta en el sistema por la misma empresa.
- **Editar producto:** El usuario empresa puede editar todos los campos previamente mencionados en la creación del producto.
- **Eliminar producto:** El usuario empresa puede eliminar cualquier producto previamente creado.

- **Gestión de rutas:** Todo usuario empresa tiene acceso a una tabla con todas las rutas creadas, desde esta tabla puede:
 - **Crear una nueva ruta** El usuario empresa puede crear nuevas rutas, incluyendo la siguiente información sobre estas:
 - Nombre de la ruta.
 - El canal de recepción.
 - La zona de mercado (desde que regiones se puede emplear esa ruta, por ejemplo, Barcelona).
 - El punto de recepción (Donde se recibe el producto).
 - Que productos están asociados a esta ruta.
 - **Editar ruta:** El usuario empresa puede editar todos los campos previamente mencionados en la creación de la ruta.
 - **Eliminar ruta:** El usuario empresa puede eliminar cualquier ruta previamente creada.

- **Cambiar el plan:** El usuario empresa puede cambiar el plan en cualquier momento (los tres planes son - free, basic y premium) (al cambiar el plan el usuario ha de realizar el pago correspondiente).

6.3. Diagrama Conceptual del Dominio

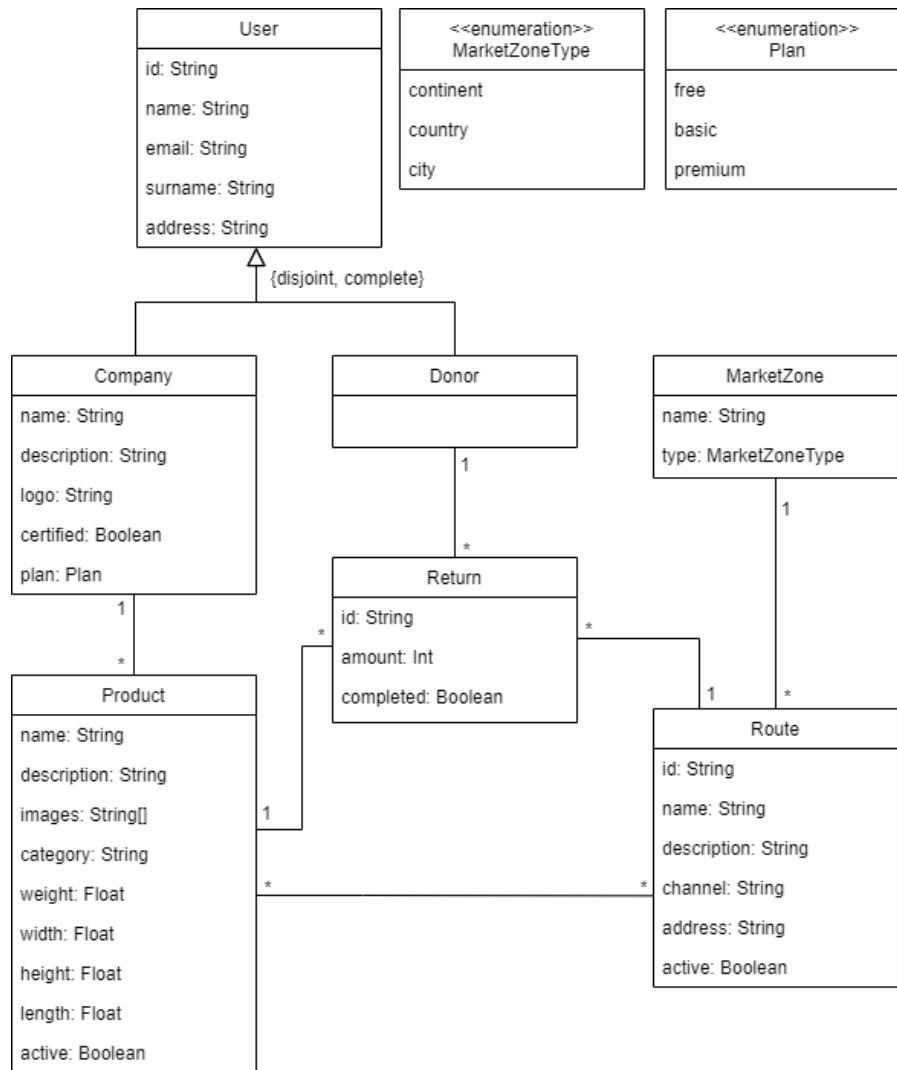


Figura 5: Diagrama de clases UML. Fuente: elaboración propia.

- RT1 - Claves externas** (User, id) (Company, User:id), (Donor, User:id), (Product, name + User:id), (Route, id), (MarketZone, name + type), (Return, id).
- RT2** - La route de un return ha de ser del mismo product que se está retornando.
- RT3** - El atributo name no se puede repetir entra las Companies.
- RT4** - El atributo name no se puede repetir entra los Products de una misma Company.

El diagrama de la figura 5 describe la estructura de datos del sistema, detallando todas las entidades involucradas en la ejecución de los diferentes casos de uso.

Se puede observar como hay dos tipos de **users**: usuarios **company** (empresa) y usuarios **donor** (donantes). Un usuario empresa tiene distintos **products** (productos), los cuales se pueden asociar a diferentes **routes** (rutas). una ruta indica a donde se entrega el producto (**route:address**) y desde donde se puede enviar (**marketZone** - una **marketZone** equivale a una región del mundo, puede ser un continente, país o ciudad). Un usuario **donor** (donante) puede realizar un **return** (retorno) de un cierto **product** usando cierta **route**.

7. Diseño

7.1. Arquitectura del Sistema

La arquitectura implementada para el desarrollo del sistema del proyecto Revivack es una arquitectura basada en microservicios. La arquitectura de software basada en microservicios es un enfoque de diseño y desarrollo de software donde una aplicación se construye como una colección de servicios pequeños e independientes que se comunican entre sí. En el caso de esta aplicación, dichos servicios se ejecutan en instancias de Amazon EC2 y son gestionados por Docker, que administra la ejecución de los contenedores.

A continuación, se muestra un diagrama resumen para mayor entendimiento de la arquitectura del sistema:

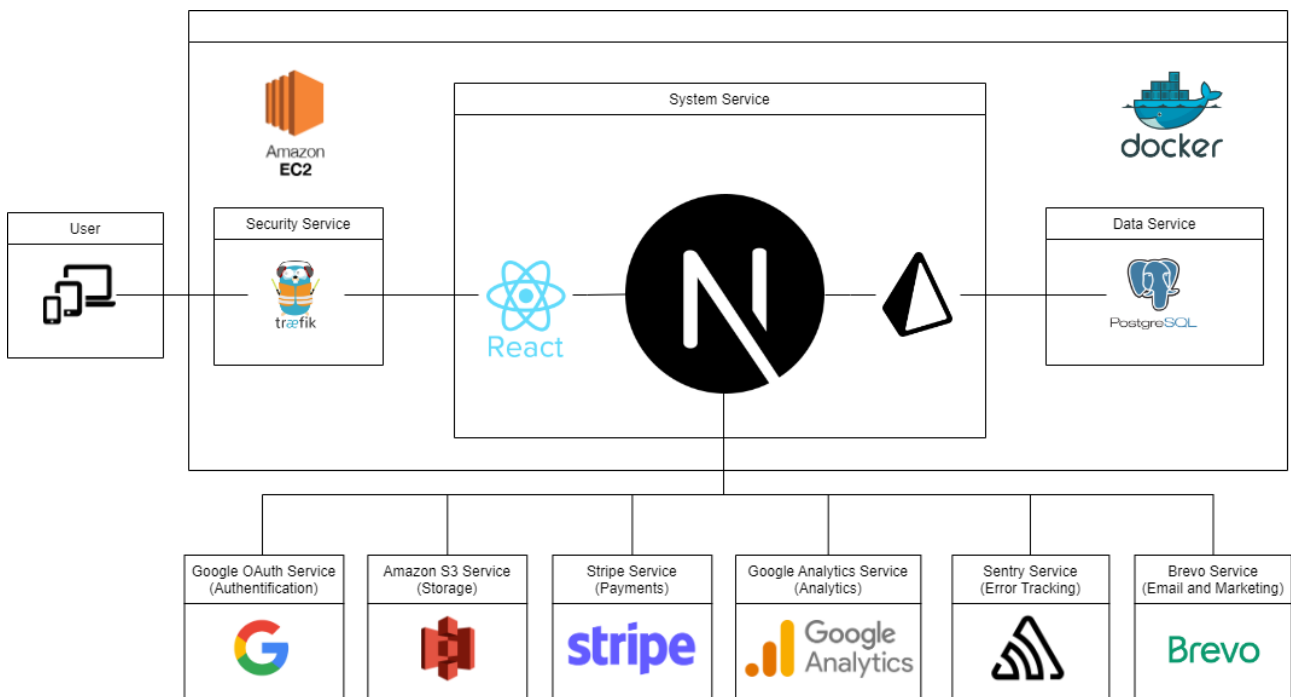


Figura 6: Arquitectura del sistema. Fuente: elaboración propia.

Como se puede observar en la figura 6, los tres servicios principales de la aplicación son:

- **Security Service (Traefik)**
 - Funciona como un proxy inverso y balanceador de carga.
 - Gestiona el enrutamiento de solicitudes HTTP y HTTPS hacia la aplicación Next.js (System Service).
 - Expone los puertos 80 (HTTP) y 443 (HTTPS) al mundo exterior y redirige el tráfico a los servicios internos según las reglas definidas.

■ System Service (Next.js)

- Sirve la aplicación web mediante React.
- Se comunica con el servicio de base de datos PostgreSQL (Data Service) utilizando Prisma como ORM⁷ (Object-Relational Mapping).
- Depende de Traefik para recibir y gestionar el tráfico HTTP/HTTPS.
- Interactúa con varios servicios externos para funcionalidades específicas:
 - **Autenticación:** Utiliza Google para la autenticación de usuarios.
 - **Almacenamiento en la nube:** Utiliza Amazon S3 para almacenar y recuperar archivos.
 - **Procesamiento de pagos:** Integra con Stripe para manejar pagos y suscripciones.
 - **Análisis:** Implementa Google Analytics para rastrear y analizar el tráfico del sitio.
 - **Monitoreo de errores:** Utiliza Sentry para rastrear y gestionar errores en la aplicación.
 - **Correo Electrónico:** Se conecta a Brevo para crear, enviar y programar correos electrónicos.

■ Data Service (PostgreSQL)

- Proporciona la base de datos relacional para almacenar los datos de la aplicación.
- Se comunica directamente con Prisma para gestionar las consultas de la base de datos.

Cabe destacar que cada uno de estos servicios está contenido en un contenedor de Docker.

⁷Es una técnica de programación para convertir datos entre el sistema de tipos utilizado en un lenguaje de programación orientado a objetos y la utilización de una base de datos relacional como motor de persistencia (Atencio 2016).

7.2. Diseño de la Seguridad (Security Service)

Como se puede observar en la figura 6, el Security Service está compuesto exclusivamente por ciertas funcionalidades de la herramienta Traefik. Traefik es un proxy inverso y balanceador de carga moderno diseñado para gestionar la entrada y salida de tráfico HTTP y HTTPS en aplicaciones basadas en contenedores (Traefik 2024).

Función de Traefik

- **Proxy Inverso:** Traefik actúa como intermediario para las solicitudes de los clientes a los servidores backend. Redirige las solicitudes entrantes (de los usuarios) a los servicios internos correspondientes.
- **Balanceador de Carga:** Distribuye el tráfico entre múltiples instancias de un servicio, mejorando la disponibilidad y la capacidad de respuesta de la aplicación.

Puertos Abiertos

En el caso de esta aplicación, Traefik abre dos puertos:

- **Puerto 80 (HTTP):** Traefik abre el puerto 80 para manejar tráfico HTTP no seguro re-dirigiendo este tráfico al puerto 443 (HTTPS) para asegurar que todas las comunicaciones se realicen de forma segura.
- **Puerto 443 (HTTPS):** Traefik abre el puerto 443 para manejar tráfico HTTPS, que es el estándar para comunicaciones seguras en la web. HTTPS cifra los datos transmitidos entre los clientes y el servidor, protegiendo la integridad y confidencialidad de la información.

En esta aplicación, Traefik gestiona el tráfico hacia los servicios internos (System Service y Data Service). Traefik enruta las solicitudes hacia el System Service (Next.js) basado en reglas de enrutamiento configuradas en el archivo `docker-compose.yml`.

7.3. Diseño del Sistema (System Service)

Como se puede observar en la figura 6, el System Service está principalmente desarrollado en Next.js. Next.js soporta tanto el renderizado de componentes en el lado del cliente (CSR) como el renderizado en el lado del servidor (SSR), utilizando las **server actions** (explicado posteriormente en la sección de *Tecnologías Empleadas*) para comunicar los componentes de cliente con el servidor.

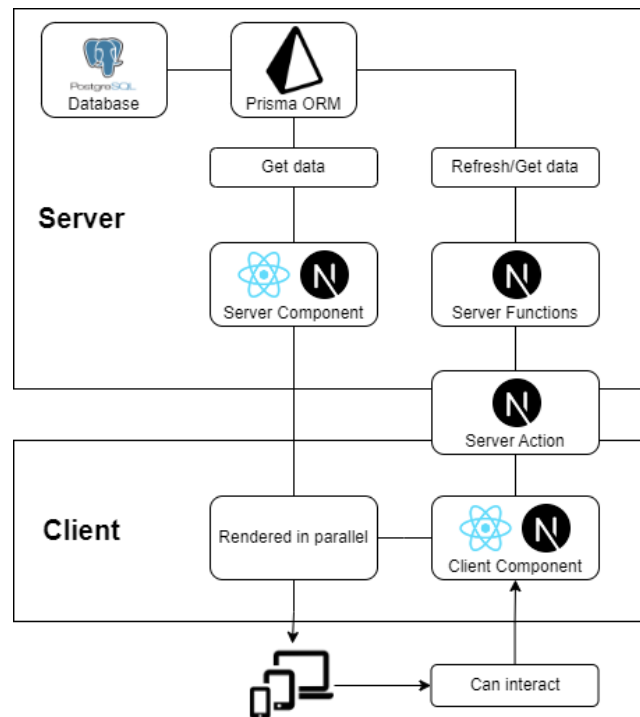


Figura 7: Funcionamiento de separación de componentes entre cliente y servidor. Fuente: elaboración propia.

Como se puede observar en la figura 7 el usuario solo puede interactuar con los componentes renderizados en el cliente aunque tiene la posibilidad de visualizar ambos. Al interactuar con un componente de cliente, este puede acceder al servidor mediante el uso de las **server actions**.

7.3.1. Tecnologías Empleadas

Para mayor entendimiento del System Service de la figura 7, se explican las tecnologías empleadas en este servicio.

7.3.1.1 Next JS

Next.js es un framework para la creación de aplicaciones web que ofrece una estructura adicional, características y optimizaciones que permiten construir aplicaciones más rápido y sin

necesidad de configurar herramientas de manera manual (Documentation 2024). A continuación, se detallan las principales características y el funcionamiento de Next.js:

Principales Características de Next.js

- **Ruteo basado en el sistema de archivos:** Next.js utiliza el sistema de archivos para definir las rutas de la aplicación. Esto facilita la creación de rutas al simplemente agregar archivos y carpetas dentro del directorio `src`.
- **Renderizado:** Soporta tanto el Renderizado en el Lado del Cliente (CSR) como el Renderizado en el Lado del Servidor (SSR), permitiendo elegir la mejor estrategia según las necesidades de la aplicación.
- **Obtención de Datos:** Ofrece soporte para `async/await` en los componentes de React, facilitando la obtención de datos de manera asíncrona y eficiente, así como el uso de las `server actions` (Explicado posteriormente).
- **Estilos:** Next.js soporta varios métodos de estilización, incluyendo módulos CSS, Tailwind CSS y CSS-in-JS, ofreciendo flexibilidad en la gestión de estilos.
- **Optimización:** Incluye optimizaciones automáticas para imágenes, fuentes y scripts, mejorando el rendimiento de la aplicación.

Estructura del Proyecto NextJS

Next.js organiza el proyecto en varias carpetas y archivos (Las carpetas y archivos deben llamarse exactamente como se indica a continuación para que la aplicación funcione correctamente) de alto nivel:

Carpetas de nivel superior:

- `app`: Enrutador de la aplicación (carpeta base).
- `public`: Activos estáticos a ser servidos (como imágenes por ejemplo).
- `src`: Carpeta opcional de código fuente de la aplicación.

Archivos de nivel superior:

- `page`: UI específica para una ruta (para la ruta `https://www.revivack.com/dashboard` por ejemplo).
- `layout`: UI compartida entre múltiples rutas.

- **route**: API endpoint para una ruta específica.

Las carpetas se usan para definir rutas. Una ruta puede ser un camino de carpetas anidadas, con una jerarquía de sistema de archivos con cada camino incluyendo un archivo `page.js` o `page.tsx`. Por ejemplo, la página `https://www.revivack.com/dashboard/products` contiene el archivo `page.js` dentro de la carpeta `products` la cual está dentro de la carpeta `dashboard`. En el caso de querer utilizar una carpeta que no sea parte de la ruta (por motivos de organización por ejemplo), esta se ha de escribir entre paréntesis (**options**) por ejemplo. Si se desea una interfaz de usuario que se comparta entre `dashboard` y `products`, se debe crear un archivo `layout.js` o `layout.tsx` en la carpeta `dashboard`, indicando así que se comparte entre `dashboard` y todas las carpetas por debajo, en este caso `products`.

Los archivos se utilizan para crear la UI para cada segmento de ruta (pages, layouts, componentes, etc).

Enrutamiento Dinámico

Los segmentos dinámicos permiten crear rutas basadas en datos dinámicos que se rellenan en el tiempo de solicitud o se prerenderizan en el tiempo de compilación. Se crean envolviendo el nombre de una carpeta entre corchetes: `[folderName]`. Ejemplos: `[id]` o `[slug]`.

Los segmentos dinámicos se pasan con el prop `params` a la página de diseño y al sistema de archivos de la ruta.

```
export default function Page({ params }: { params: { slug: string } }) {  
  return <div>My Post: {params.slug}</div>  
}
```

Server Actions

Para gestionar los componentes renderizados en el cliente, las **server actions** en Next.js permiten ejecutar código del lado del servidor en respuesta a eventos del cliente, como clics de botones o envíos de formularios, y luego actualizar la interfaz de usuario en consecuencia. Esto es útil para manejar lógica compleja o interactuar con bases de datos y otros servicios del lado del servidor desde componentes renderizados en el cliente sin tener que escribir una API completa. Es similar a una simulación de una API, permitiendo una integración más sencilla y directa de la lógica del servidor en la aplicación.

Estilización y SEO

Next.js admite diferentes formas de estilizar una aplicación, incluyendo:

- **CSS global:** Archivo css compartido por toda la aplicación.
- **Módulos CSS:** Módulos css utilizados por componentes específicos.
- **CSS-in-JS:** CSS directamente dentro de los archivos Javascript.

Además, ofrece una API de meta-datos para mejorar el SEO⁸ y la compatibilidad web, generando automáticamente los elementos `<head>` relevantes para las páginas.

7.3.1.2 React JS

React es una biblioteca de JavaScript de código abierto diseñada para construir interfaces de usuario (UI) de manera eficiente y flexible. Fue desarrollada por Facebook y lanzada por primera vez en 2013. React permite a los desarrolladores crear aplicaciones web dinámicas y rápidas al facilitar la creación de componentes reutilizables y modulares (Muñoz 2023).

Componentes en React

Los componentes son la base de React. Un componente es una pieza independiente y reutilizable de la interfaz de usuario. Puede variar desde un botón a un formulario entero. Los componentes permiten dividir la UI en partes más pequeñas y manejables. Aquí se pueden observar los dos tipos de componentes en React:

- **Componentes de Clase:** Son componentes creados usando clases de ES6 (ECMAScript 6: un estándar creado con el objetivo de permitir la interoperabilidad de diferentes páginas web entre distintos navegadores (Hackademy 2023)). Estos componentes pueden tener estado y métodos del ciclo de vida.

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

⁸El SEO es un conjunto de acciones orientadas a mejorar el posicionamiento de un sitio web en la lista de resultados de buscadores de internet.

- **Componentes Funcionales:** Son componentes creados usando funciones. Gracias a los Hooks, estas funciones pueden manejar estado y efectos secundarios (React 2024).

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Hooks

Los Hooks permiten usar estado y otras características de React sin escribir una clase. Los Hooks proporcionan una manera más directa y simple de manejar el estado y otros efectos en los componentes funcionales (React 2024). Aquí se pueden ver los principales hooks de react:

- **useState:** Permite añadir estado a los componentes funcionales:

```
import React, { useState } from 'react';  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>You clicked {count} times</p>  
      <button onClick={() => setCount(count + 1)}>  
        Click me  
      </button>  
    </div>  
  );  
}
```

- **useEffect:** Permite realizar efectos secundarios en componentes funcionales, como operaciones de datos, suscripciones o cambios manuales del DOM (explicado posteriormente en la sección de *Virtual DOM*):

```
import React, { useEffect } from 'react';

function Example() {
  useEffect(() => {
    document.title = `You clicked ${count} times`;
  });

  return <div />;
}
```

- **useContext:** Permite acceder al contexto de React en componentes funcionales:

```
import React, { useContext } from 'react';
const ThemeContext = React.createContext('light');

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button theme={theme}>I am styled by theme context!</button>;
}
```

Virtual DOM

El Virtual DOM (Document Object Model: es una API definida para representar e interactuar con cualquier documento HTML o XML (Contributors 2023)). Es una representación en memoria del DOM real. Cuando el estado de un componente cambia, React actualiza el Virtual DOM primero, compara las diferencias con el DOM real (proceso conocido como reconciliación) y luego realiza solo las actualizaciones necesarias. Esto hace que las aplicaciones React sean más eficientes y rápidas.

Gestión de los Estados

React proporciona varias formas de manejar el estado en las aplicaciones:

- **Estado Local:** Usando `useState` en componentes funcionales o `this.state` en componentes de clase.
- **Estado Global:** Usando contextos (`useContext`) o bibliotecas externas como Redux o MobX para manejar el estado compartido entre múltiples componentes.

- **Estado Derivado:** Usando `useMemo` o `useCallback` para memorizar valores o funciones que dependen de otros estados o props.

7.3.1.3 Prisma

Prisma es una herramienta ORM (Object-Relational Mapping) para bases de datos. Está diseñada para simplificar el acceso y manejo de bases de datos en aplicaciones Node.js y TypeScript. Prisma facilita la interacción con bases de datos proporcionando una API tipada (enteros, strings, booleanos, etc.) para realizar consultas y modificaciones (Prisma 2024). Prisma es compatible con varias bases de datos, incluyendo PostgreSQL, MySQL, SQLite y SQL Server.

Componentes de Prisma

Prisma consta de varias herramientas para facilitar el desarrollo de base de datos. Las herramientas principales son:

- **Prisma Client:** Prisma Client es una biblioteca de consultas generada automáticamente para realizar operaciones con bases de datos. Proporciona una API para crear, leer, actualizar y eliminar datos en la base de datos. En el siguiente ejemplo se puede observar un `get` de usuarios:

```
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

async function main() {
  const allUsers = await prisma.user.findMany();
  console.log(allUsers);
}
```

- **Prisma Migrate:** Prisma Migrate es una herramienta de migración⁹ de base de datos que facilita la gestión del esquema de la base de datos a lo largo del ciclo de vida de la aplicación. Permite crear y aplicar migraciones.
- **Prisma Studio:** Prisma Studio es una herramienta visual para explorar y editar datos en la base de datos. Proporciona una interfaz gráfica para visualizar y manipular los datos, facilitando el desarrollo y la depuración.

⁹Es el proceso de transferir datos de una o varias bases de datos de origen a una o más bases de datos de destinos (Fatima 2024).

- **Esquema de Prisma:** El esquema de Prisma es un archivo de configuración que define el modelo de datos, las fuentes de datos (bases de datos) y los generadores (como Prisma Client). El archivo `schema.prisma` define la estructura de la base de datos (del cual se genera el SQL automáticamente).

7.3.2. Diseño de las Pantallas

En el System Service se renderiza la aplicación para ser ofrecida a los usuarios. A continuación, usando como guía los casos de uso, se enseñan las distintas pantallas de la aplicación, logrando así una mayor comprensión del flujo de la aplicación (es importante señalar que se muestran las pantallas implementadas para ordenador, aunque también se han implementado versiones para teléfono móvil y tableta).

A partir de los casos de uso presentados en la figura 2 (todo tipo de usuarios), se muestran las pantallas correspondientes a las funcionalidades indicadas en dichos casos de uso.

Registrarse

En las figuras 8 y 9 se pueden observar las dos pantallas principales que el usuario utiliza para registrarse:

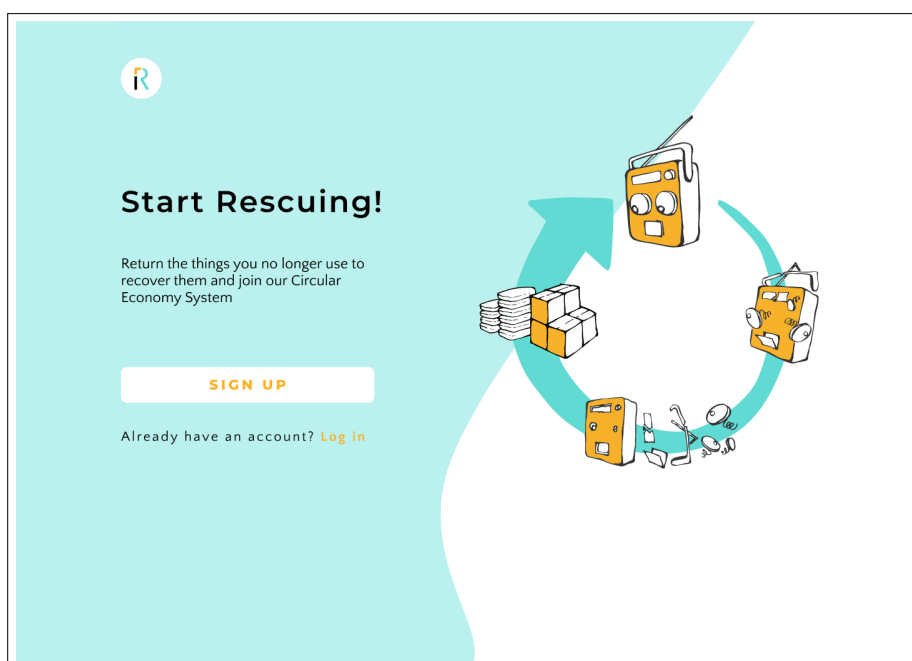


Figura 8: Pantalla principal de la aplicación. Fuente: elaboración propia.

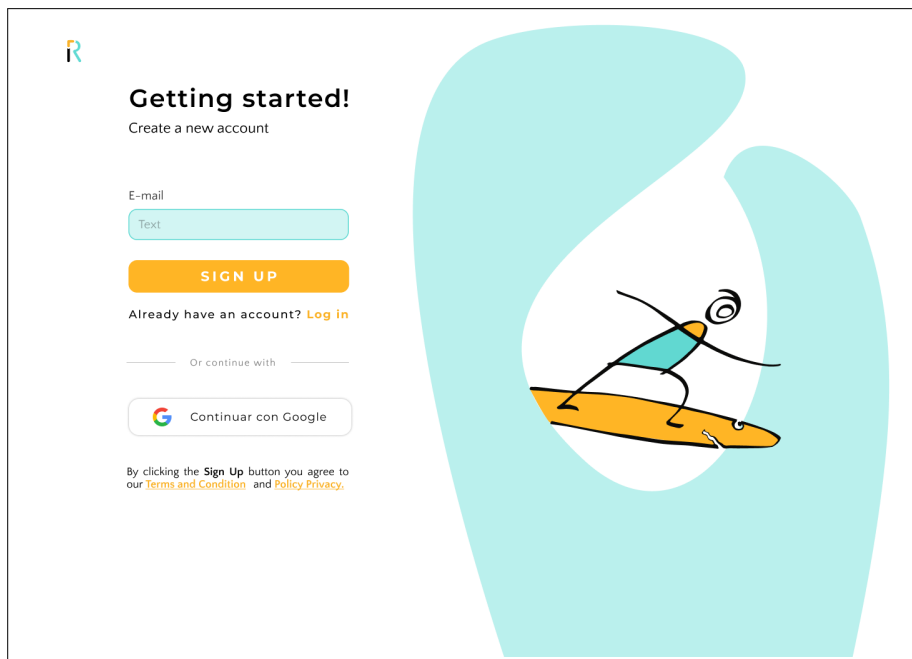


Figura 9: Pantalla de registro de la aplicación. Fuente: elaboración propia.

En la figura 9 se puede observar como el usuario puede registrarse tanto con cualquier email (haciendo uso del MagicLink antes mencionado) o directamente mediante la autenticación de Google.

Después de introducir el email, se le hacen ciertas preguntas al usuario para proceder con el proceso de registro:

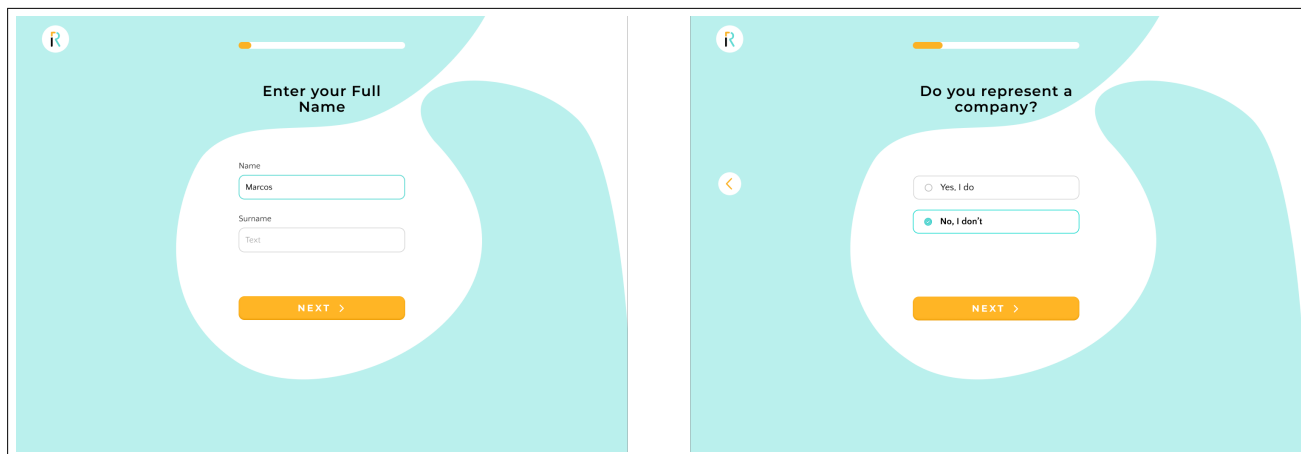


Figura 10: Pantalla del primer paso del proceso de registro de la aplicación. Fuente: elaboración propia.

Como se puede observar en la figura 10 después de introducir el nombre el usuario ha de indicar si es una empresa o un donante, en el caso de ser una empresa, este introduce la información requerida en la figura 11 y en el caso de ser un donante la solicitada en la figura 12:

What kind of company are you?

Product manufacturers & marketers
I design and/or manufacture products, directly or through others, to be sold.

Consultants
I am not directly involved in the return and recovery flow, but my role is important because I advise and activate.

Other entities
I do not do whatever of the mentioned activities, but I interested in getting involved.

Shops
I have a store or network of stores where I sell products.

Recovery Centers
I play a role in the recovery chain of items or components.

Logistic company
I am a shipping and warehousing company.

Company Information
Product manufacturers & marketers
Own Brand & Shop

Company Name
Popsicase

Identify by
Select Options

Identification number
Text

NEXT >

In which country is your head office located?
Product manufacturers & marketers
Own Brand & Shop

Country
Select Options

Address
Text

Zip Code
Text

City
Select Options

NEXT >

What's your job position?
Product manufacturers & marketers
Own Brand & Shop

Role
Text

NEXT >

Figura 11: Pantalla de registro de la aplicación en caso de empresa. Fuente: elaboración propia.

Después de rellenar la información solicitada observada en la figura 11 (tipo de empresa, nombre y número de identificación, ubicación y rol en la empresa) el usuario empresa ya está registrado y puede acceder al sistema con todas las funcionalidades.

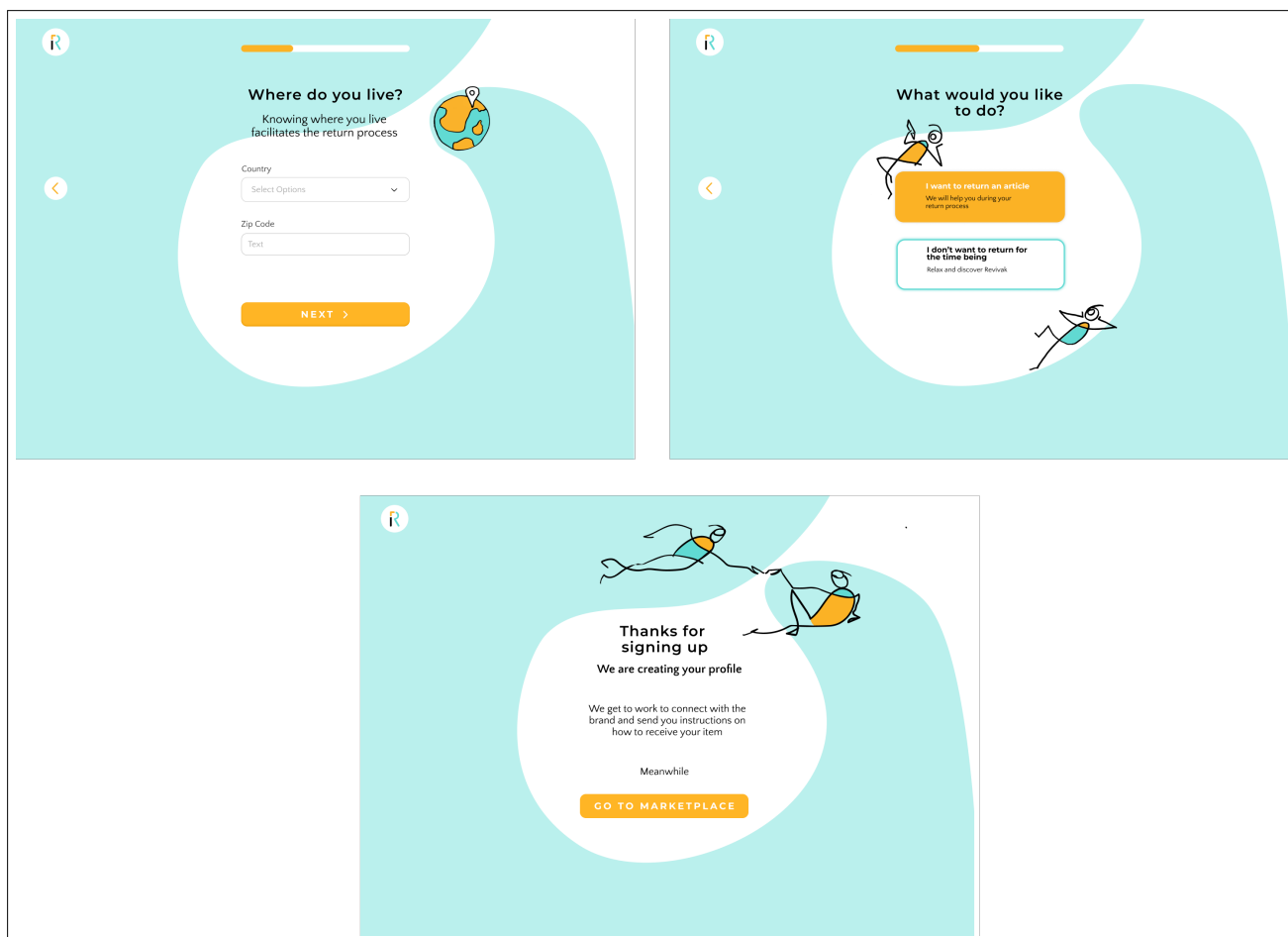


Figura 12: Pantalla de registro de la aplicación en caso de donante. Fuente: elaboración propia.

Como se puede observar en la figura 12 una vez indicada la ubicación del usuario, este puede decidir si sugerir productos de retorno a modo de información para Revivack o visitar directamente el sistema (botón de Marketplace).

Iniciar sesión

En la figura 14 se puede ver la pantalla que utiliza el usuario para iniciar sesión (de la misma forma que en la pantalla de registro (figura 9) el usuario se puede iniciar sesión tanto con el MagicLink como directamente con Google):

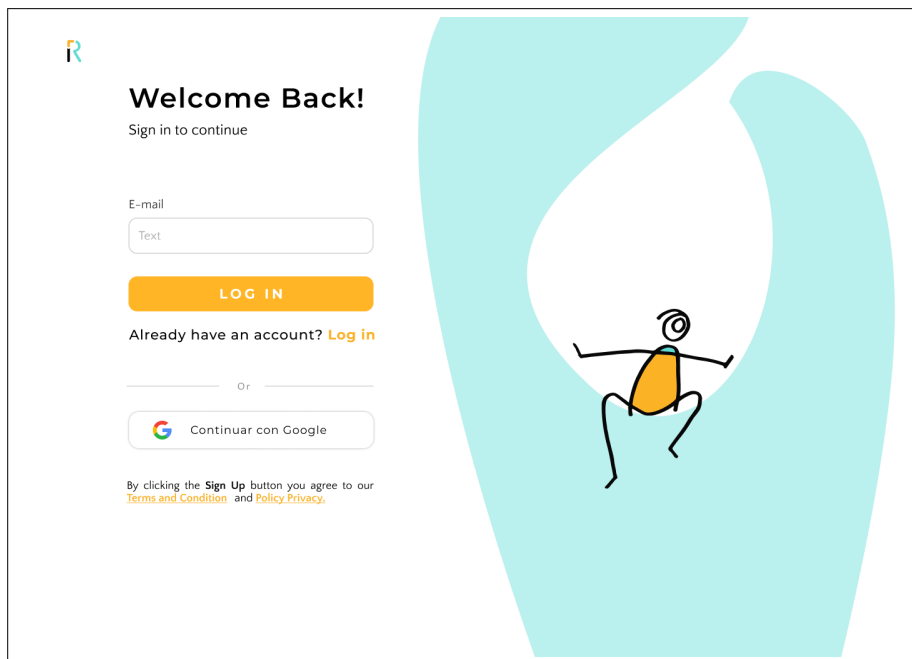


Figura 13: Pantalla de inicio de sesión de la aplicación. Fuente: elaboración propia.

En el caso de ser un usuario empresa este accederá directamente al dashboard del sistema y en el caso de ser un usuario donante a la siguiente pantalla:

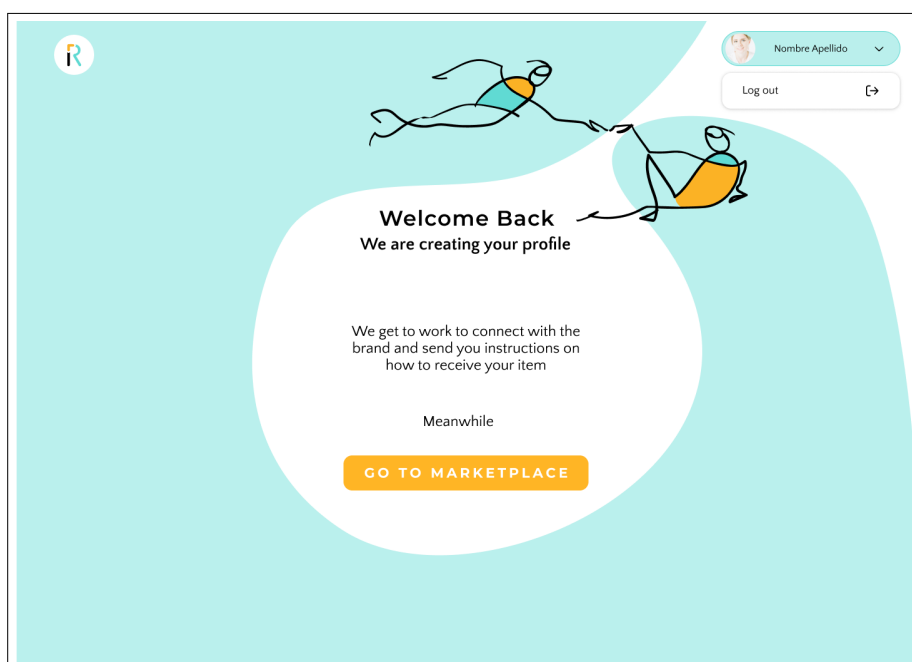


Figura 14: Pantalla de la aplicación en caso de usuario donante. Fuente: elaboración propia.

Cerrar sesión y Cambiar idioma

En la figura 15 se puede observar el componente que se reutiliza en todas las pantallas del sistema para poder cerrar sesión, de manera similar al componente para cambiar el idioma que se muestra en la figura 16:

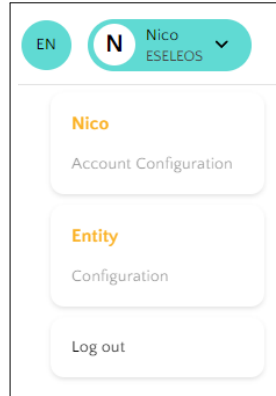


Figura 15: Componente para cerrar sesión. Fuente: elaboración propia.



Figura 16: Componente para cambiar de idioma. Fuente: elaboración propia.

Ver producto

En la figura 17 se puede observar la pantalla donde se muestra el perfil de una empresa, accesible iniciando o no sesión. Nótese como, en este caso, no hay ningún producto publicado (en el apartado de products de la figura 17); en esa sección se verían mostrados los diferentes productos publicados:

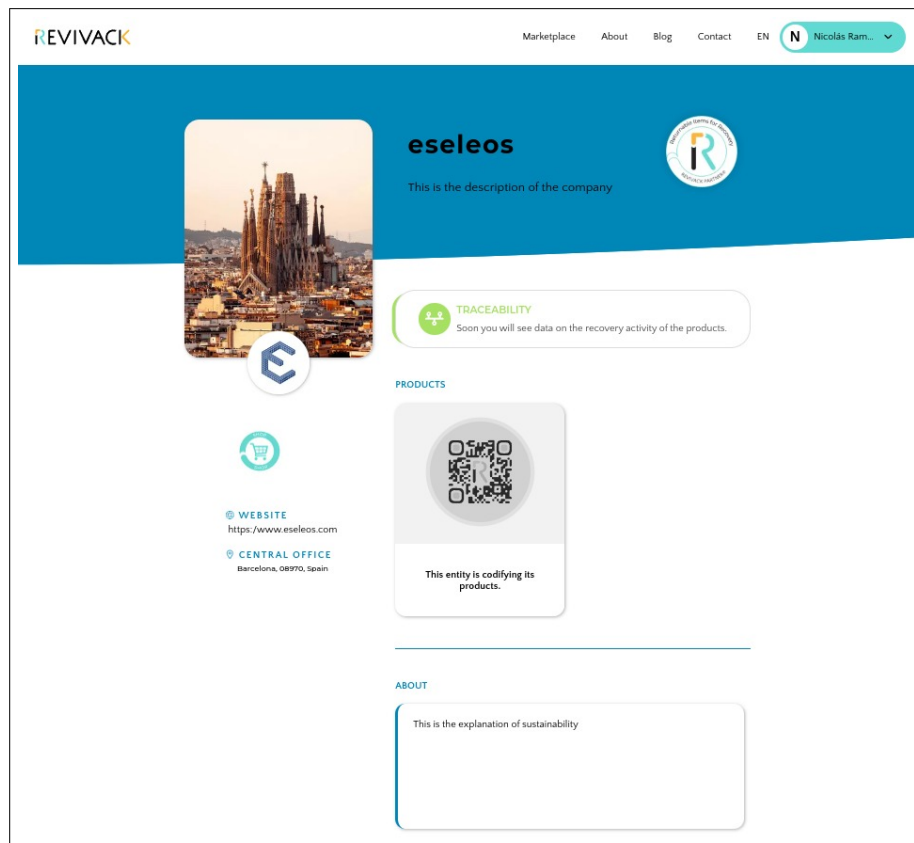


Figura 17: Pantalla de visualización del perfil de una empresa. Fuente: elaboración propia.

Ver producto

En la figura 18 se puede observar la pantalla donde se muestra un producto de una empresa, accesible iniciando o no sesión:

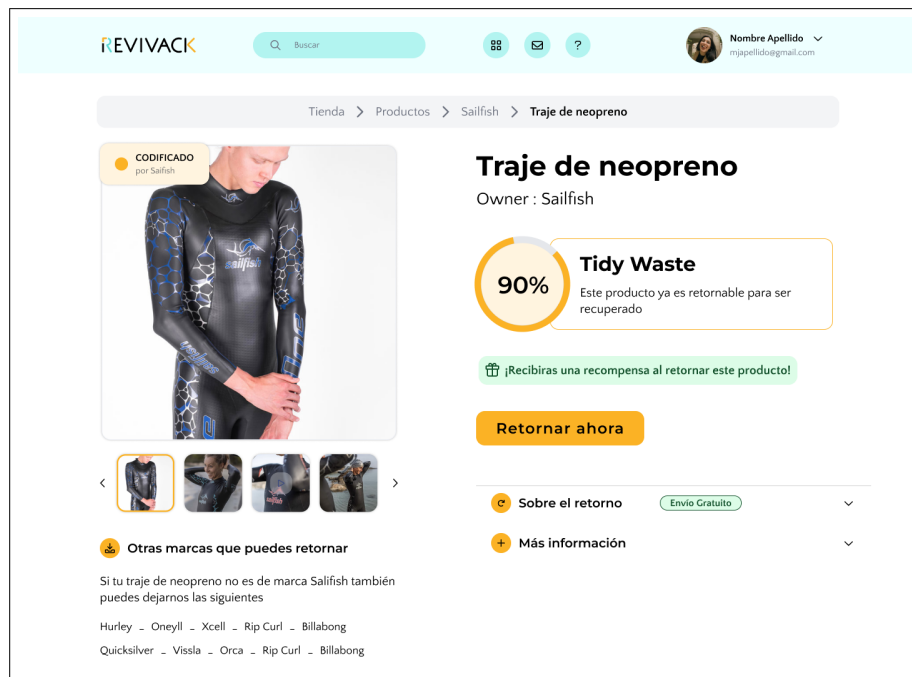


Figura 18: Pantalla de visualización de un producto. Fuente: elaboración propia.

A partir de los casos de uso de la figura 4 (usuarios empresa), se muestran las pantallas correspondientes a las funcionalidades descritas en dichos casos de uso.

Dashboard

Como se puede observar en la figura 19 la aplicación incluye una topbar y una sidebar fijas que se pueden utilizar en todo momento por el usuario empresa. La topbar permite al usuario cambiar de plan, acceder al marketplace, cambiar de idioma, cerrar sesión y pedir ayuda en todo momento (ayuda mediante whatsapp, email, solicitando una llamada o accediendo a las FAQ), la sidebar permite al usuario empresa acceder al resto de funcionalidades del sistema (perfil, productos y rutas) (nótese como el resto de funcionalidades están deshabilitadas tanto en la sidebar como en el dashboard ya que son funcionalidades a desarrollar a futuro):

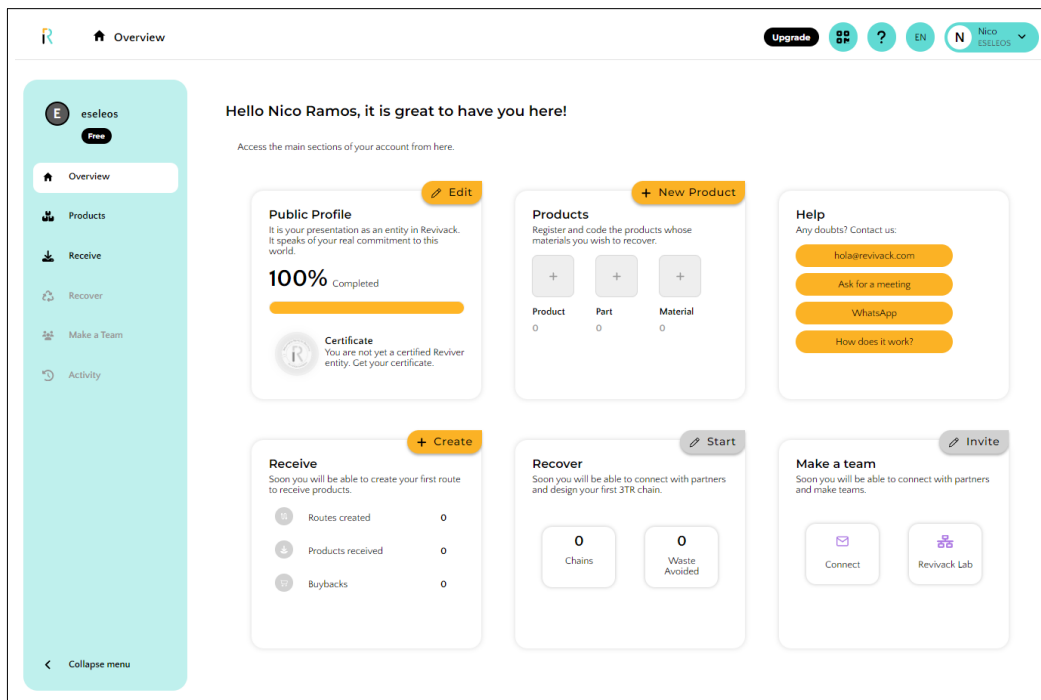


Figura 19: Pantalla del dashboard de la aplicación. Fuente: elaboración propia.

Como se había mencionado en los casos de uso, el usuario empresa tiene información general de su perfil, en la figura 19 se puede ver de que forma el usuario accede a esa información. Cabe destacar también que haciendo click en cada uno de los cuadros de información se puede también acceder a dichas funcionalidades (menos las deshabilitadas).

Edición del perfil

En las figuras 20 y 21 se pueden ver los distintos campos que el usuario puede modificar de su perfil, así como la posibilidad de previsualizarlo (nótese en la figura 21 como los apartados de return y recover están deshabilitados). Para que el usuario pueda publicar su perfil por primera vez, hay ciertos campos que son obligatorios: logo, imagen corporativa y descripción:

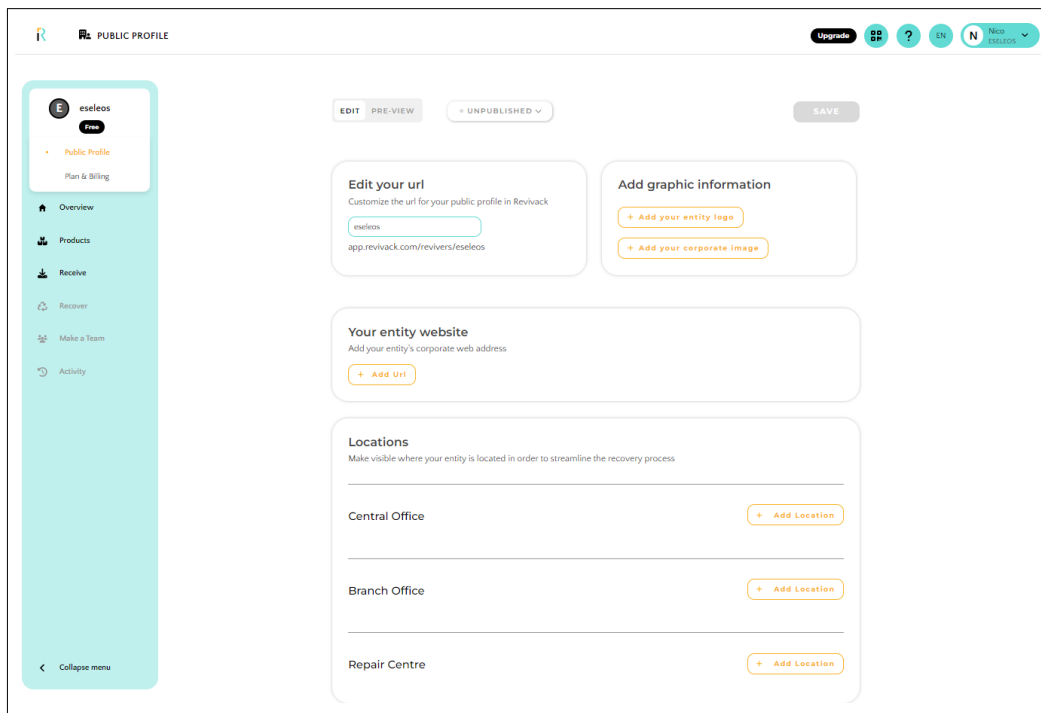


Figura 20: Pantalla de edición de perfil. Fuente: elaboración propia.

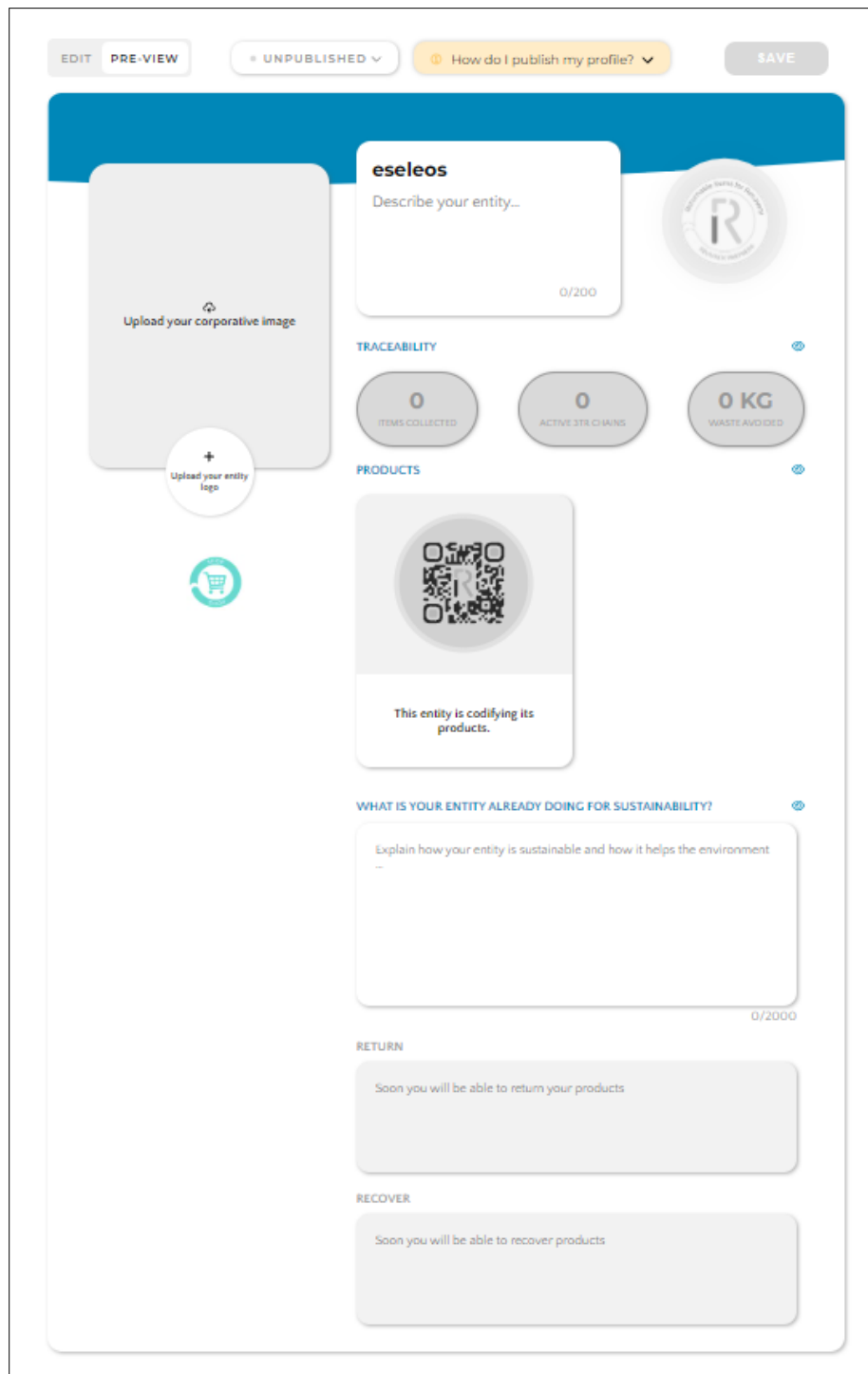


Figura 21: Componente de previsualización del perfil público. Fuente: elaboración propia.

Gestión de producto y eliminar producto

En la figura 22 se puede ver la tabla con todos los productos del usuario, pudiendo filtrarlos, eliminarlos y acceder a editarlos:

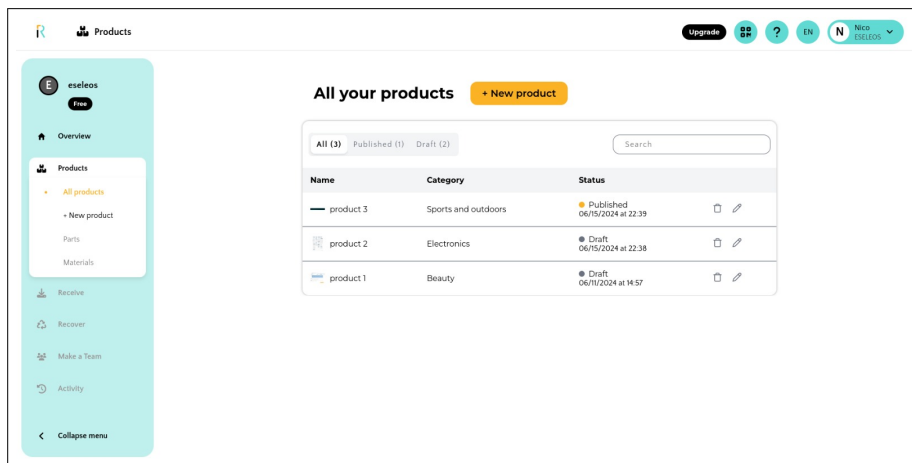


Figura 22: Pantalla de gestión de productos. Fuente: elaboración propia.

Crear y editar producto

En la figura 23 se pueden ver todos los campos que el usuario ha de rellenar para crear/editar un producto, cabe destacar que todos estos campos son obligatorios para crear/publicar un producto:

Crear y editar ruta

En la figura 24 se pueden observar todos los campos que el usuario ha de rellenar para crear/editar una ruta, cabe destacar que todos estos campos son obligatorios para crear/activar una ruta:

The screenshot displays a web application interface for creating and editing a route. The layout consists of a sidebar on the left and a main content area. The sidebar menu includes 'Overview', 'Products' (with 'New Product' highlighted), 'Receive', 'Recover', 'Make a Team', and 'Activity'. The main content area features a top navigation bar with 'Free', 'Buscar', and user profile information. Below this is a 'Ruta Inactiva' toggle switch. The primary form is titled 'Detalles de la Ruta' and contains four sections: 'Nombre de la Ruta' (text input), 'Canal de recepción' (dropdown), 'Zona de Mercado' (dropdown with '+ Crear Nueva' button), and 'Producto asociado' (multi-select with '+ Crear Nuevo' button). At the bottom of this section are 'Guardar' and 'Siguiente' buttons. The second section, 'Dirección de la Ruta', is titled 'Desk / Punto de Recepción' and includes a 'Dirección Propia' / 'Partner' toggle, a search bar, and a form for 'Nombre del Desk' (text input), 'Country' (dropdown), 'Zip Code' (text input), 'Address' (text input), and 'City' (text input). A 'Guardar' button is at the bottom right of this section.

Figura 24: Pantalla de creación/edición de ruta. Fuente: elaboración propia.

Cambiar plan

En la figura 26 se puede observar como el usuario empresa puede escoger a que plan cambiarse, así como ver las ventajas que cada uno de estos planes tiene. Este componente es un popup que aparece al hacer clic en el botón de 'Cambiar plan'. Al hacer clic en alguno de los planes, el usuario ha de realizar el pago mediante Stripe:

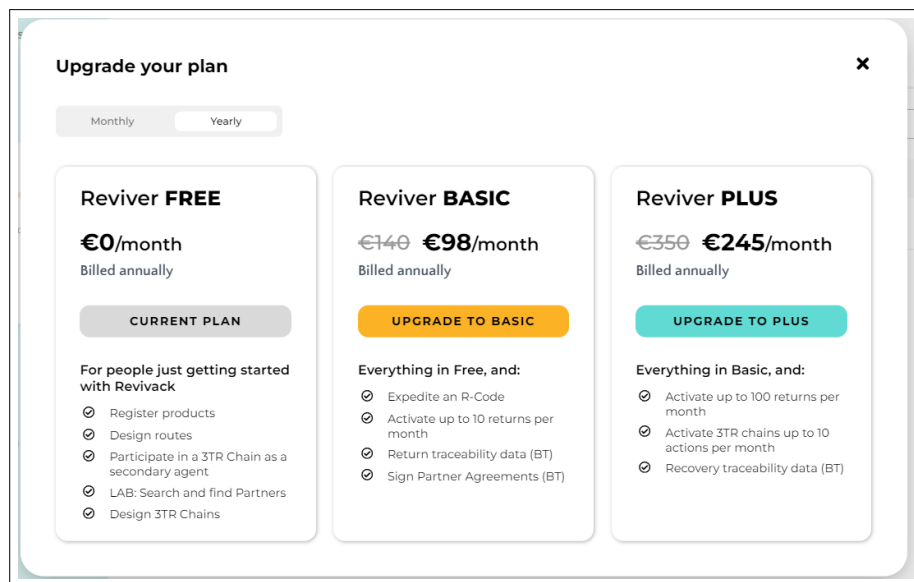


Figura 25: Componente (popup) de cambio de plan. Fuente: elaboración propia.

A partir de los casos de uso de la figura 3 (usuarios donante), se muestran las pantallas correspondientes a las funcionalidades indicadas en dichos casos de uso.

Retornar producto

Al hacer clic en el botón de retornar al visualizar el producto de una empresa (figura 18), aparece el popup de la figura para realizar el retorno del producto siguiendo los tres pasos indicados. Cabe destacar que, en el apartado de '¿Cómo quieres enviarlo?' aparecerán las rutas que tenga activas ese producto y que la ubicación indicada en '¿Desde donde quieres retornarlo?' este dentro de la zona de mercado¹⁰ de esa ruta:

¹⁰Zona para cada ruta que indica desde donde se puede emplear dicha ruta.

Figura 26: Tres pasos del componente (popup) de retorno de un producto. Fuente: elaboración propia.

7.3.3. Patrones de Diseño

A continuación, se presentan los principales patrones de diseño utilizados durante el desarrollo de la aplicación:

- **Component-Based:** El patrón Component-Based se centra en la construcción de la interfaz de usuario mediante la creación de componentes independientes y reutilizables. Cada componente encapsula su propia lógica, estilos y estructura, lo que permite que estos se desarrollen, mantengan y prueben de manera aislada. Los componentes pueden ser combinados para formar interfaces de usuario más complejas. Este enfoque promueve la modularidad, la reutilización de código y la mantenibilidad, facilitando la gestión de la aplicación. Este patrón se aplica básicamente solo al crear y reusar componentes.
- **Observer Pattern:** El patrón Observer establece una relación de uno a muchos entre objetos, donde un objeto conocido como el sujeto mantiene una lista de otros objetos llamados observadores. Estos observadores se suscriben al sujeto para recibir notificaciones de cambios en su estado. Cuando el estado del sujeto cambia, notifica automáticamente a todos sus observadores, permitiendo que estos reaccionen adecuadamente. En el caso de esta aplicación, se ha empleado, por ejemplo, con el uso de `events listeners` en ciertos components (figura 27):

```

useEffect(() => {
  const handleResize = () => {
    setScreenSize({
      width: window.innerWidth,
      height: window.innerHeight,
    });
  };

  window.addEventListener("resize", handleResize);

  return () => {
    window.removeEventListener("resize", handleResize);
  };
}, []);

```

Figura 27: Uso del observer pattern con un Event Listener. Fuente: elaboración propia.

- **Singleton Pattern:** El patrón Singleton asegura que una clase tenga solo una instancia y proporciona un punto de acceso global a esa instancia. Se utiliza generalmente para gestionar recursos compartidos, como configuraciones, registros o conexiones a bases de datos, donde tener múltiples instancias podría llevar a estados inconsistentes o al uso ineficiente de recursos. En el case de esta aplicación, se ha empleado, por ejemplo, para gestionar la instancia de `PrismaClient` (figura 28):

```

import { PrismaClient } from "@prisma/client";

const prismaClientSingleton = () => {
  return new PrismaClient({
  })
}

declare global {
  var client: undefined | ReturnType<typeof prismaClientSingleton>
}

export const client = globalThis.client ?? prismaClientSingleton()

if (process.env.NODE_ENV !== 'production') globalThis.client = client

```

Figura 28: Uso del singleton pattern con Prisma Client. Fuente: elaboración propia.

La función `prismaClientSingleton` de la figura 28 crea una nueva instancia de `PrismaClient`. El objeto `client` se asigna utilizando `globalThis.client ?? prismaClientSingleton()`, lo que verifica si `globalThis.client` ya contiene una instancia. Si no, crea una nueva instancia. Esto asegura que solo exista una instancia de `PrismaClient` en toda la aplicación.

7.4. Diseño de la Base de Datos (Data Service)

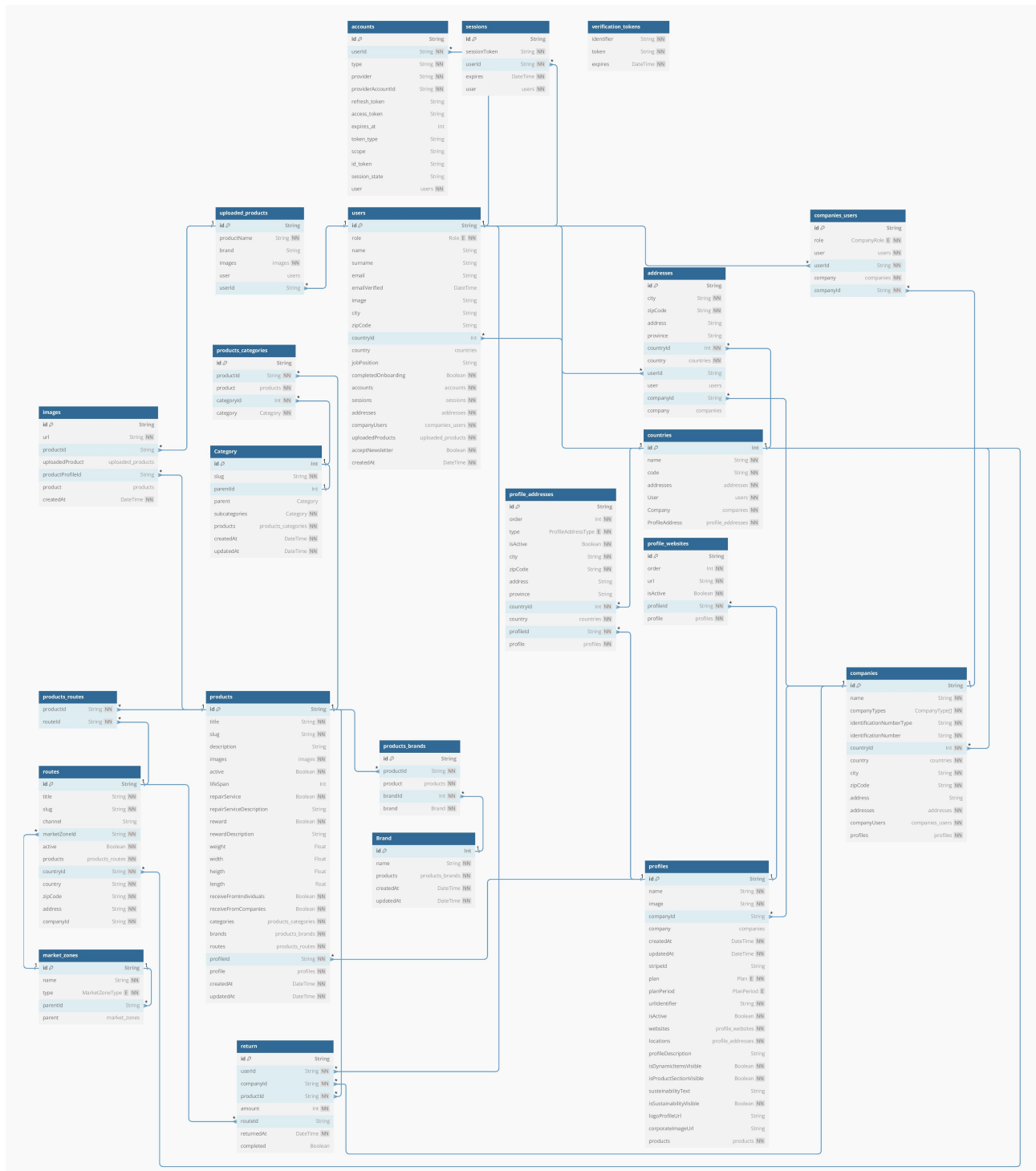


Figura 29: Estructura de la base de datos - Diagrama Entidad Relación. Fuente: elaboración propia.

En la figura 29 se puede observar la estructura de la base de datos, diseñada para gestionar un sistema que abarca usuarios, productos, empresas y sus diversas interacciones. En cuanto al diagrama como tal, la tabla **users** almacena información personal y de contacto, incluyendo

roles, direcciones y datos de perfil. Las tablas **accounts** y **sessions** gestionan la autenticación y las sesiones de los usuarios, garantizando la seguridad y el seguimiento de las actividades. La tabla **companies** contiene detalles sobre las empresas, como sus tipos, números de identificación y ubicaciones, mientras que la tabla **profiles** maneja la información de los perfiles empresariales, incluyendo el plan (free, basic o plus) y la visibilidad de diferentes secciones.

Los productos están registrados en la tabla **products**, la cual incluye su título, descripción, dimensiones, y estados de actividad, junto con relaciones a sus **categorías**, **marcas** y **rut**as. La tabla **uploaded_products** gestiona los productos subidos por los usuarios, vinculándolos con imágenes almacenadas en la tabla **images**. Además, la base de datos maneja la verificación de tokens a través de la tabla **verification_tokens** para asegurar la autenticidad de las operaciones.

Las relaciones entre tablas incluyen referencias y eliminaciones en cascada para mantener la integridad de los datos. Las tablas como **products_routes**, **products_categories**, y **products_brands** establecen relaciones entre productos y sus respectivas **rut**as, **categorías** y **marcas**. Las direcciones y ubicaciones se gestionan a través de las tablas **addresses** y **profile_addresses**, vinculadas a **usuarios** y **empresas**.

8. Implementación

8.1. Implementación de la Arquitectura

Como se ha mencionado anteriormente (ver figura 6 de la sección *Arquitectura del Sistema*), esta aplicación utiliza una arquitectura basada en microservicios.

Para lograr implementar esta estructura, se hace uso del siguiente archivo `docker compose`:

```
1 services:
2   traefik:
3     image: traefik:v2.2
4     container_name: traefik
5     restart: unless-stopped
6     command:
7       - "--providers.docker=true"
8       - "--providers.docker.exposedbydefault=false"
9       - "--providers.docker.network=revivack-external"
10      - "--entrypoints.web.address=:80"
11      - "--entrypoints.websecure.address=:443"
12      - "--certificatesresolvers.myresolver.acme.httpchallenge=true"
13      - "--certificatesresolvers.myresolver.acme.httpchallenge.entrypoint=
web"
14      #- "--certificatesresolvers.myresolver.acme.caserver=https://acme-
staging-v02.api.letsencrypt.org/directory"
15      - "--certificatesresolvers.myresolver.acme.email=sysadmin@eseleos.com"
16      - "--certificatesresolvers.myresolver.acme.storage=/letsencrypt/acme.
json"
17     ports:
18       - "0.0.0.0:80:80"
19       - "0.0.0.0:443:443"
20     volumes:
21       - "/var/run/docker.sock:/var/run/docker.sock:ro"
22       - traefik:/letsencrypt
23     labels:
24       - "traefik.enable=true"
25       - "traefik.http.routers.redirect.rule=HostRegexp(`{host:.+}`)"
26       - "traefik.http.routers.redirect.entrypoints=web"
27       - "traefik.http.routers.redirect.middlewares=redirect-to-https"
28       - "traefik.http.middlewares.redirect-to-https.redirectscheme=
https"
```

```

29     networks:
30         - external
31
32 nextjs:
33     image: ${NEXT_DOCKER_TAG?Variable not set}
34     container_name: nextjs
35     restart: unless-stopped
36     environment:
37         - GOOGLE_CLIENT_ID=${GOOGLE_CLIENT_ID?Variable not set}
38         - GOOGLE_CLIENT_SECRET=${GOOGLE_CLIENT_SECRET?Variable not set}
39         - LINKEDIN_CLIENT_ID=${LINKEDIN_CLIENT_ID?Variable not set}
40         - LINKEDIN_CLIENT_SECRET=${LINKEDIN_CLIENT_SECRET?Variable not set}
41         - EMAIL_SERVER=${EMAIL_SERVER?Variable not set}
42         - EMAIL_FROM=${EMAIL_FROM?Variable not set}
43         - NEXTAUTH_SECRET=${NEXTAUTH_SECRET?Variable not set}
44         - NEXTAUTH_URL=https://${APP_URL?Variable not set}/
45         - DB_HOST=${DB_HOST?Variable not set}
46         - DATABASE_URL=${DATABASE_URL?Variable not set}
47         - S3_BUCKET_NAME=${S3_BUCKET_NAME?Variable not set}
48         - S3_REGION=${S3_REGION?Variable not set}
49         - S3_ACCESS_KEY=${S3_ACCESS_KEY?Variable not set}
50         - S3_SECRET_KEY=${S3_SECRET_KEY?Variable not set}
51         - BREVO_API_KEY=${BREVO_API_KEY?Variable not set}
52         - SENTRY_ENVIRONMENT=${ENV?Variable not set}
53         - NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=${
54     NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY?Variable not set}
55         - STRIPE_SECRET_KEY=${STRIPE_SECRET_KEY?Variable not set}
56         - STRIPE_WEBHOOK_SECRET=${STRIPE_WEBHOOK_SECRET?Variable not set}
57         - GOOGLE_ANALYTICS_ID=${GOOGLE_ANALYTICS_ID?Variable not set}
58     labels:
59         - "traefik.enable=true"
60         - "traefik.http.routers.nextjs.rule=Host(`${APP_URL?Variable not set
61     }`)"
62         - "traefik.http.routers.nextjs.entrypoints=websecure"
63         - "traefik.http.routers.nextjs.tls.certresolver=myresolver"
64     depends_on:
65         - postgres
66     networks:
67         - external
68         - internal

```

```

67
68 postgres:
69     image: postgres:13.2
70     restart: unless-stopped
71     volumes:
72         - db-data:/var/lib/postgresql/data
73     ports: ["0.0.0.0:$DB_PORT:5432"]
74     environment:
75         POSTGRES_USER: ${DB_USER?Variable not set}
76         POSTGRES_PASSWORD: ${DB_PASS?Variable not set}
77         POSTGRES_DB: ${DB_NAME?Variable not set}
78     networks:
79         - internal
80
81 networks:
82     external:
83         name: "revivack-external"
84     internal:
85         name: "revivack-internal"
86
87 volumes:
88     traefik:
89     db-data:

```

Figura 30: Docker compose de la aplicación. Fuente: elaboración propia.

En el código de la figura 30 se puede ver como se declaran los tres contenedores (**traefik** línea 2, **nextjs** línea 32 y **postgres** línea 69) y como el resto de servicios externos se comunican con **nextjs** mediante las distintas variables (API keys, servidores, ids, etc.) declaradas desde la línea 37 a la línea 56.

8.2. Implementación del Sistema (System Service)

Cabe destacar que para desarrollar el sistema se ha empleado el lenguaje TypeScript¹¹.

8.2.1. Librerías Externas

A continuación se listan las diferentes librerías externas utilizadas para la implementación de esta aplicación, incluyendo una breve explicación de para que se han empleado:

- **Auth.js (@auth/prisma-adapter@1.0.0)**: Se ha usado para facilitar la integración con Prisma, más específicamente para manejar la autenticación de usuarios en la aplicación, proporcionando funciones como registro, inicio de sesión y manejo de sesiones que interactúan con la base de datos.
- **AWS (@aws-sdk/client-s3@3.427.0)**: Se ha usado para interactuar con el servicio S3 (Simple Storage Service) de AWS (Amazon Web Services). Se ha empleado para gestionar el almacenamiento en la nube, permitiendo subir, descargar y administrar archivos en 'buckets' de S3.
- **Format.js (@formatjs/intl-localematcher@0.4.0)**: Es una librería que sirve para la internacionalización (i18n) de aplicaciones. Se ha empleado para formatear mensajes, fechas y números en diferentes 'locales' (configuraciones regionales), haciendo que la aplicación sea accesible en múltiples idiomas.
- **Brevo (@getbrevo/brevo@2.0.0-beta.4)**: Se ha usado para enviar correos electrónicos, SMS y manejar campañas de marketing. También se ha empleado para la automatización de marketing y gestión de comunicaciones con los clientes.
- **Hook Form (@hookform/resolvers@3.3.4)**: Este paquete **resolvers** es una extensión para React Hook Form, una librería para la gestión de formularios en React. Se ha empleado para la integración con validadores externos "Zod"(explicado posteriormente), facilitando la validación de formularios.
- **Radix-ui**: Esta librería incluye una colección de componentes accesibles y estilizados para React, componentes como:

¹¹TypeScript es un lenguaje de programación fuertemente tipado que se basa en JavaScript y le brinda mejores herramientas a cualquier escala (TypeScript 2024).

- **@radix-ui/react-checkbox@1.0.4**: Se ha utilizado para implementar 'checkboxes'.
 - **@radix-ui/react-collapsible@1.0.3**: Se ha empleado para crear secciones colapsables.
 - **@radix-ui/react-dialog@1.0.5**: Se ha utilizado para implementar diálogos modales.
 - **@radix-ui/react-dropdown-menu@2.0.6**: Se ha usado para crear menús desplegables.
 - **@radix-ui/react-label@2.0.2**: Se ha empleado para añadir etiquetas en formularios.
 - **@radix-ui/react-popover@1.0.7**: Se ha utilizado para implementar 'popovers' (desplegables).
 - **@radix-ui/react-switch@1.0.3**: Se ha usado para implementar interruptores.
 - **@radix-ui/react-tabs@1.0.4**: Se ha empleado para crear pestañas.
 - **@radix-ui/react-tooltip@1.0.7**: Se ha utilizado para implementar 'tooltips' (descripción emergente).
- **React email**: Es un conjunto de herramientas para crear y manejar correos electrónicos en aplicaciones React.
 - **@react-email/components@0.0.14**: Se ha utilizado para crear componentes reutilizables para los correos electrónicos.
 - **@react-email/preview@0.0.9**: Se ha empleado para previsualizar los correos electrónicos.
 - **@react-email/render@0.0.12**: Se ha usado para renderizar los correos electrónicos.
 - **Sentry-next.js (@sentry/nextjs@8.2.1)**: Es una librería para la monitorización de errores en aplicaciones Next.js. Sentry se ha empleado para capturar y reportar errores y excepciones, facilitando el 'debugging' y mejorando la estabilidad de la aplicación.
 - **Slugify (@sindresorhus/slugify@2.2.1)**: Slugify sirve para convertir cadenas de texto

en slugs¹² amigables para la URL. Esto se ha empleado para crear URLs limpias y SEO-friendly basadas en títulos o nombres.

- **Stripe (@stripe/stripe-js@3.3.0)**: Stripe sirve para manejar pagos en línea. Se ha empleado para integrar fácilmente funcionalidades de procesamiento de pagos en la aplicación.
- **TypeScript (typescript@5.2.2)**: Es un superconjunto de JavaScript que añade tipado estático. Se ha empleado para mejorar la mantenibilidad y escalabilidad del código.
 - **@types/negotiator@0.6.1**: Se ha utilizado para proporcionar tipos¹³ a la librería negotiator.
 - **@types/node@20.6.5**: Se ha empleado para proporcionar tipos a Node.js.
 - **@types/nodemailer@6.4.14**: Se ha utilizado para proporcionar tipos a Nodemailer.
 - **@types/react-dom@18.2.6**: Se ha empleado para proporcionar tipos a React DOM.
 - **@types/react@18.2.14**: Se ha utilizado para proporcionar tipos a React.
- **Date-fns (date-fns@3.6.0)**: Sirve para manejar fechas en JavaScript. Se ha empleado para manipular, formatear y calcular fechas.
- **Eslint (eslint@8.44.0)**: Es una librería que se emplea como una herramienta de 'linting' para JavaScript y TypeScript. Se ha usado para analizar el código en busca de problemas de estilo y errores potenciales.
- **Negotiator (negotiator@0.6.3)**: Es una librería para la para la 'negociación' de contenido en HTTP. Se ha empleado para determinar el mejor formato de respuesta (JSON, XML, etc.) basado en las cabeceras de la solicitud.
- **Next-auth (next-auth@4.22.1)**: Es una solución de autenticación para aplicaciones. Se ha empleado para soportar múltiples proveedores de autenticación como OAuth.
- **Next-intl (next-intl@3.4.4)**: Es una librería de internacionalización para Next.js. Se ha empleado para facilitar la traducción de aplicaciones Next.js a múltiples idiomas.

¹²El slug es una pequeña parte de un enlace que permite identificar un contenido específico en una web (Koller 2023).

¹³Un tipo es una manera conveniente de referirse a las diferentes propiedades y funciones que tiene un valor.

- **Next (next@13.5.6)**: Se ha usado como un framework, el cual que facilita la creación de aplicaciones web con renderizado del lado del servidor (SSR) y generación estática de sitios (SSG) entre otras cosas. (como explicado anteriormente).
- **Nodemailer (nodemailer@6.9.9)**: Es una herramienta que sirve para enviar correos electrónicos desde aplicaciones Node.js. Se ha empleado para soportar múltiples servicios de correo.
- **Prisma (prisma@5.10.2)**: Se ha empleado para simplificar la interacción con bases de datos, esta librería proporciona una API intuitiva para realizar consultas y mutaciones. (como explicado anteriormente).
 - **@prisma/client@5.10.2**: Se ha utilizado como el cliente de Prisma para interactuar con la base de datos.
- **React (react@18.2.0)**: Se ha usado para construir interfaces de usuario. (como explicado anteriormente).
 - **react-dom@18.2.0**: Se ha empleado como el complemento de React para trabajar con el DOM.
 - **react-dropzone@14.2.3**: Se ha usado para crear componentes con carga de archivos mediante arrastrar y soltar.
 - **react-email@2.1.3**: Se ha empleado para manejar correos electrónicos en React.
 - **react-hook-form@7.51.0**: Se ha utilizado para gestionar formularios en React con hooks.
 - **react-hot-toast@2.4.1**: Se ha usado para implementar notificaciones ('toasts').
 - **react-icons@4.11.0**: Se ha empleado para añadir diversos iconos en los diferentes componentes.
 - **react-select@5.8.0**: Se ha usado como un componente de selección.
- **Sharp (sharp@0.33.2)**: Es una librería que sirve para el procesamiento de imágenes. Se ha empleado para redimensionar, convertir y manipular imágenes de manera eficiente.
- **Stripe (stripe@15.4.0)**: Es otra versión de la librería de Stripe para procesamiento de pagos.

- **Server Only (server-only@0.0.1)**: Se ha usado para asegurar que cierto código solo se ejecute en el servidor, muy útil para Next.js.
- **Vitest (vitest@1.5.3)**: Se ha usado como un framework de testing para el proyecto.

A parte de todas estas librerías también se ha hecho uso del software Docker. Docker es una plataforma de virtualización a nivel de sistema operativo ('software containerization platform' en inglés). Una plataforma de este tipo permite crear una aplicación y empaquetarla junto con sus dependencias y librerías en un contenedor o **container**, como se conocen habitualmente (Blanch 2024).

En el caso de este proyecto, Docker se ha utilizado para:

- Desarrollar en local y levantar una base de datos.
- Construir una imagen que se pueda desplegar en diferentes entornos.

8.2.2. Traducciones

como se ha comentado anteriormente, para implementar la aplicación en dos idiomas se ha hecho uso de la librería `next-intl`, más concretamente la función `useTranslations()`. La forma en la que funciona la traducción de la aplicación es la siguiente:

- Se definen dos archivos json (`en.json` y `es.json`) con la misma estructura pero cambiando el contenido de cada elemento. como por ejemplo:

<pre> }, "UserRepresentationPage": { "pleaseSelect": "Please select an option", "company?1": "Do you represent a", "company?2": " entity?", "yesCompany": "Yes, I do", "noCompany": "No, I don't" }, </pre>	<pre> 61 62 63 64 65 66 67 68 </pre>	<pre> }, "UserRepresentationPage": { "pleaseSelect": "Por favor, selecciona una opción", "company?1": "¿Representas a una", "company?2": " entidad?", "yesCompany": "Sí", "noCompany": "No" }, </pre>
---	--------------------------------------	---

Figura 31: Json de idiomas (izquierda `en.json` y derecha `es.json`). Fuente: elaboración propia.

- La primera ruta de la aplicación equivale al idioma empleado, utilizando una ruta dinámica de Next (encapsulando la carpeta en `[]`) se indica que idioma se está utilizando, en el caso de español: `app.revivack.com/es` y en el caso de inglés `app.revivack.com/en`.
- En los diferentes componentes se define una variable para después poder acceder a uno de estos dos archivos jsons usando `next-intl` dependiendo de la ruta dinámica. Siguiendo con el ejemplo de la figura 31:

```
import { useTranslations } from "next-intl";

export default function UserRepresentationPage() {

  const t = useTranslations();

  return (
    <div className={styles.Container}>
      <h2 className={styles.Title}>
        {t('UserRepresentationPage.company?1')}<br></br>{t('UserRepresentationPage.company?2')}
      </h2>
    </div>
  );
}
```

Figura 32: Uso de traducciones en un componente. Fuente: elaboración propia.

Implementación cliente-servidor

Como se ha mencionado anteriormente, la aplicación de Revivack utiliza principalmente Next.js para el desarrollo del sistema, lo que permite separar los componentes entre el renderizado del cliente y del servidor, como se muestra en la figura 7 de la sección *Diseño del Sistema (System Service)*:

A continuación, se muestra como se ha implementado la estructura de la figura 7, dando ejemplos sobre diferentes componentes y como algunos de ellos se comunican con el resto del sistema.

8.2.3. Server Components

A continuación se puede ver un ejemplo de un server component (pantalla de nuevo producto, figura 23). Como se puede observar en la figura 33, el código empieza directamente por los `imports`, lo cual le indica a next que es un server component (si no habría un `'use client'` indicando que es un componente de cliente o un `'use server'` indicando que es una `server action`):

```

import { auth } from "@lib/auth";
import { getCategories } from "@lib/categories";
import { getProfileIdByUserId } from "@lib/profile";
import { ProductForm } from "../product-form";

export default async function ProductsNewPage() {
  const categories = await getCategories();
  const session = await auth();

  const profileId = await getProfileIdByUserId({
    userId: session?.user?.userId ?? "",
  });

  if (!profileId) {
    return <div>No profile</div>;
  }

  return (
    <div>
      <ProductForm
        profileId={profileId.id}
        profileSlug={profileId.name}
        categories={categories}
        mode="create"
      />
    </div>
  );
}

```

Figura 33: Archivo page de la página de new product (server component). Fuente: elaboración propia.

Como se puede observar en la figura 33, este componente simplemente muestra otro componente llamado `productForm`, el cual es un client component; esta práctica de incluir client components dentro de server components es muy común y optimiza el renderizado de la UI (en vez de poner todo directamente en un client component). En el caso de que se deseara, también se podría mostrar la información extraída de la base de datos (como por ejemplo las categorías) en una lista (siempre y cuando el usuario no interactúe con esta).

El componente se comunica directamente con la base de datos utilizando las funciones de `getCategories()`, `auth()` y `getProfileIdByUserId(userId)`. Estas funciones están implementadas en la carpeta `lib` ya que son funciones que se reutilizan en distintos server components. En la figura 34 se puede ver la implementación de la función `getCategories()` para ver como se comunica con el Data Service a través de Prisma:

```

export async function getCategories({
  parent,
}: { parent?: number } = {}): Promise<TypeCategory[]> {
  // Resulting struct
  const categoriesWithSubcategories = [];

  // Fetch the categories with the given parent ID
  const categories = await client.category.findMany({
    where: {
      parentId: parent ?? null,
    },
  });

  for (const category of categories) {
    const subcategories = await getCategories({ parent: category.id });

    categoriesWithSubcategories.push({
      id: category.id,
      slug: category.slug,
      subCategories: subcategories,
    });
  }

  return categoriesWithSubcategories;
}

```

Figura 34: Función de obtener las categorías de la base de datos (accedida por server components). Fuente: elaboración propia.

Otro claro ejemplo de la implementación de un server component es la pantalla con la tabla de productos (figura 22).

```

type Filter = "all" | "draft" | "published";

export default async function Products({
  searchParams,
}): {
  searchParams?: {
    search?: string;
    page?: string;
    filter?: Filter;
  };
} {
  const filter: Filter = searchParams?.filter || "all";
  const search = searchParams?.search || "";

  const t = await getTranslations("SettingsProductPage");
  const tCategory = await getTranslations("Categories");

  const session = await auth();
  const profileId = await getProfileIdByUserId({
    userId: session?.user?.userId ?? "",
  });

  const products = await client.product.findMany({
    where: {
      profileId: profileId?.id,
      title: {
        contains: search,
        mode: "insensitive",
      },
      active: filter === "all" ? undefined : filter === "draft" ? false : true,
    },
    include: {
      images: true,
      categories: {
        include: {
          category: true,
        },
      },
    },
    orderBy: {
      createdAt: "desc",
    },
  });
}

```

Figura 35: Archivo page de la página products (server component) - uso de searchParams.
Fuente: elaboración propia.

La diferencia entre componente el componente de la figura 35 y el anterior (figura 33), es que este logra comunicarse con un client component hijo (**ProductTableFilters**) a través de la URL. En Next.js, un client component hijo se comunica con un server component padre utilizando los **searchParams** (los **searchParams** se indican al final de la URL después de un '?', como en el caso de filter en este ejemplo: app.revivack.com/dashboard/products?filter=all) (Next 2024) de la URL (en este caso para filtrar y buscar productos). Como se puede observar en la figura 35, el server component padre, **Products**, recibe los **searchParams** accediendo a la URL y los utiliza para obtener y filtrar productos desde la base de datos.

En la figura 36 se puede ver como este componente renderiza una tabla de productos con un client component hijo, **ProductTableFilters**, que permite al usuario cambiar los filtros y realizar búsquedas (explicado en la sección de *Client Components*).

```

return (
  <div className={styles.container_table_products}>
    <div className={styles.title_table_flex}>
      <h1>{t("TablePage.TableTitle")}</h1>
      <Link href="/dashboard/products/new">
        <Button
          text={t("TablePage.TableNewProduct")}
          variant="primary"
          size="regular"
          disabled={false}
        />
      </Link>
    </div>
    <div className={styles.table_products_with_filters}>
      <div className={styles.filter_search_container}>
        <ProductTableFilters
          filter={filter}
          productsQuantityByStatus={{ ...
        }}
        />
      </div>
      <table className={styles.table}>...
    </table>
    </div>
  </div>
);

```

Figura 36: Archivo page de la página products (server component) - renderizado. Fuente: elaboración propia.

8.2.4. Client Components

En las figuras 37 y 38 se puede ver el client component (véase el 'use client' de la figura 37) de los filtros de la tabla de productos (siguiendo con el ejemplo anterior):

```

"use client";

import { useTranslations } from "next-intl";
import { usePathname, useRouter, useSearchParams } from "next/navigation";
import { useDebounceCallback } from "use-debounce";
import styles from "../product-table-filters.module.css";

type Filter = "all" | "draft" | "published";

type ProductTableFiltersProps = {
  filter: Filter;
  productsQuantityByStatus: {
    total: number;
    draft: number;
    published: number;
  };
};

export default function ProductTableFilters(props: ProductTableFiltersProps) {
  const { filter, productsQuantityByStatus } = props;

  const t = useTranslations("SettingsProductPage.TablePage");
  const searchParams = useSearchParams();
  const pathname = usePathname();
  const { replace } = useRouter();

  const handleFilter = (filter: Filter) => {
    const params = new URLSearchParams(searchParams.toString());
    params.set("filter", filter);
    replace(`${pathname}?${params.toString()}`);
  };

  const handleSearch = useDebounceCallback((search: string) => {
    const params = new URLSearchParams(searchParams.toString());
    if (search) {
      params.set("search", search);
    } else {
      params.delete("search");
    }
    replace(`${pathname}?${params.toString()}`);
  }, 300);

  return (
    <div className={styles.container}>...
    </div>
  );
}

```

Figura 37: Componente de filtros la página products (client component) - lógica y uso de `searchParams`. Fuente: elaboración propia.

```

return (
  <div className={styles.container}>
    <div className={styles.navbar}>
      <div onClick={() => handleFilter("all")}>
        <p data-active={filter === "all"}>
          {` ${t("TableAllFilter")} (${productsQuantityByStatus.total})`}
        </p>
      </div>
      <div onClick={() => handleFilter("published")}>
        <p data-active={filter === "published"}>
          {` ${t("TablePublishedFilter")} (${productsQuantityByStatus.published})`}
        </p>
      </div>
      <div onClick={() => handleFilter("draft")}>
        <p data-active={filter === "draft"}>
          {` ${t("TableDraftFilter")} (${productsQuantityByStatus.draft})`}
        </p>
      </div>
    </div>
    <input
      type="text"
      placeholder={t("TableSearchPlaceholder")}
      onChange={e => handleSearch(e.target.value)}
      className={styles.input}
    />
  </div>
);

```

Figura 38: Componente de filtros la página products (client component) - renderizado. Fuente: elaboración propia.

Como se puede observar en las figuras 37 y 38, cuando el usuario interactúa con los filtros o la búsqueda de la figura 38 en `ProductTableFilters`, las funciones `handleFilter` y `handleSearch` de la figura 37 actualizan los parámetros de búsqueda en la URL usando el método `replace` del `useRouter` hook de Next.js. Estas actualizaciones en los parámetros de búsqueda desencadenan una nueva renderización del componente `Products` con los nuevos parámetros. El server component `Products` lee estos parámetros actualizados y obtiene los productos correspondientes de la base de datos, asegurando que la tabla de productos se actualice de acuerdo con los filtros seleccionados y la búsqueda realizada por el usuario (como explicado anteriormente).

El componente anterior no accede en ningún momento a la base de datos utilizando `server actions`. En la figura 39 se puede ver el componente de publicación de producto junto con su implementación en las figuras 40 y 41, componente el cual sí accede a la base de datos utilizando `server actions`:

● Published ▾

Select the state of your product

☒ **Published**
 Your product is published and available for return in the marketplace
app.revivack.com/products/eseleos_product-3

☐ **Draft**
 Your product is not published and is not available for return in the marketplace

Figura 39: Componente de publicado de un producto. Fuente: elaboración propia.

```
"use client";

import { updateProductStateAction } from "@actions/product";
import * as Popover from "@radix-ui/react-popover";
import { useTranslations } from "next-intl";
import { useState } from "react";
import { FaChevronDown } from "react-icons/fa6";
import { GoDotFill } from "react-icons/go";
import styles from "./active-product-button.module.css";

type ActivationButtonWithUrlProps = {
  uniqueUrl: string;
  profileId: string;
  productSlug: string;
  isProductPublished: boolean;
  disabled: boolean;
};

export default function ActivationButtonWithUrl(
  props: ActivationButtonWithUrlProps,
) {
  const { uniqueUrl, profileId, productSlug, isProductPublished, disabled } =
    props;

  const t = useTranslations("SettingsProductPage.ActivationButton");

  const [isLoadingStatusChange, setIsLoadingStatusChange] = useState(false);
  const [dropdownOpen, setDropdownOpen] = useState(false);

  const handleStatusChange = async (status: boolean) => {
    const hasChanged = isProductPublished !== status;
    if (!hasChanged) return;

    setIsLoadingStatusChange(true);
    const result = await updateProductStateAction({
      slug: productSlug,
      profileId: profileId,
      status: status ? "published" : "draft",
    });

    if (!result.success) {
      console.log("Error updating product status");
    }

    setIsLoadingStatusChange(false);
    setDropdownOpen(false);
  };

  return (
    <
      <Popover.Root>
      </Popover.Root>
    </>
  );
}
```

Figura 40: Componente de publicado de un producto (client component). Fuente: elaboración propia.

```

"use server";

import { auth } from "@lib/auth";
import { client } from "@lib/prisma";
import { brandSlugSchema } from "@schemas/brand";
import { productSlugSchema } from "@schemas/product";
import { IResultAction, IResultActionData } from "@types/actions";
import * as Sentry from "@sentry/nextjs";
import { revalidatePath } from "next/cache";
import * as z from "zod";

const updateProductSchema = z.object({
  slug: z.string(),
  profileId: z.string(),
  status: z.enum(["published", "draft"]),
});

type UpdateProductStateData = z.infer<typeof updateProductSchema>;

export async function updateProductStateAction(data: UpdateProductStateData): Promise<IResultAction> {
  const session = await auth();

  if (!session) {
    return {
      success: false,
      message: "User not authenticated",
    };
  }

  const result = updateProductSchema.safeParse(data);

  if (!result.success) {
    return {
      success: false,
      message: "Invalid product data",
    };
  }

  const validatedData = result.data;

  const { slug, profileId, status } = validatedData;

  try {
    await client.product.update({
      where: {
        slug_profileId: {
          slug,
          profileId,
        },
      },
      data: {
        active: status === "published",
      },
    });
  } catch (error) {
    console.log(`[SERVER ACTION] Updating product state - Error: ${error}`);
    Sentry.captureException(error);
    return {
      success: false,
      message: "Error updating product state",
    };
  }

  revalidatePath(`/dashboard/products/${slug}`);

  return {
    success: true,
    message: "Product state updated",
  };
}

```

Figura 41: Función de publicado de un producto (server action). Fuente: elaboración propia.

Como se puede observar en las figuras 41 y 40, para publicar un producto se utiliza el client component `ActivationButtonWithUrl` (observese el 'use client') junto con la server action `updateProductStateAction` (observese el 'use server'). El flujo para publicar/despublicar un producto con este componente es el siguiente:

- El componente `ActivationButtonWithUrl` es un client component que muestra un desplegable (Popover) para seleccionar el estado del producto (publicado o borrador).
- Cuando el usuario interactúa con el botón para cambiar el estado del producto, se llama a la función `handleStatusChange`. Esta función verifica si el estado actual del producto

es diferente al estado seleccionado. Si hay un cambio, se procede a actualizar el estado.

- La función `handleStatusChange` llama a la `server action updateProductStateAction` con los datos necesarios: `slug`, `profileId` y el nuevo `status` del producto.
- La `server action updateProductStateAction` se ejecuta en el servidor y realiza los siguientes pasos:
 - Autentica al usuario usando `auth`.
 - Valida los datos recibidos utilizando `zod`¹⁴ y el esquema `updateProductStateSchema`.
 - Actualiza el estado del producto en la base de datos utilizando `prisma client`.
 - Si la actualización es exitosa, se invalida la ruta de caché correspondiente con `revalidatePath` para asegurar que los cambios se reflejen en la interfaz de usuario.
- Finalmente, el componente `ActivationButtonWithUrl` actualiza su estado interno y cierra el desplegable, mostrando el nuevo estado del producto (publicado o borrador).

¹⁴Biblioteca de validación y análisis de esquemas que permite definir y validar de manera sencilla la estructura y los tipos de datos esperados en objetos (Zod 2024).

8.3. Implementación de la Base de Datos (Data Service)

Como se ha mencionado anteriormente, la base de datos de esta aplicación está en PostgreSQL y se gestiona a través de Next.js utilizando Prisma. Para crear esta base de datos, se utiliza el archivo `schema.prisma`, el cual se encarga de generar el código SQL. A continuación se incluyen algunos ejemplos de dicho archivo:

Al principio de este archivo, se indica que proveedor (postgresql) se va a utilizar junto con la url previamente mencionada en el `docker compose` (figura 30 línea 46).

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Figura 42: Datasource del archivo `schema.prisma`. Fuente: elaboración propia.

En la figura 43 se puede ver como se declara una nueva tabla de la base de datos; incluyendo el nombre, el tipo de atributo, primary key (utilizando '@id') y si hay algún tipo de relaciones (en el caso de la figura 43 se puede ver la relación con la tabla de usuarios a través de la foreign key `userId` del atributo `user` con tipo `User`).

```
model UploadedProducts {
  id          String @id @default(cuid())
  productName String
  brand       String?
  images      Image[]
  user        User?  @relation(fields: [userId], references: [id])
  userId      String?

  @@map("uploaded_products")
}
```

Figura 43: Ejemplo de creación de tabla del archivo `schema.prisma`. Fuente: elaboración propia.

También existe la posibilidad de crear un `enum`¹⁵ de la siguiente forma:

```
enum Plan {
  FREE
  BASIC
  PLUS
}
```

Figura 44: Ejemplo de `enum` del archivo `schema.prisma`. Fuente: elaboración propia.

¹⁵Un tipo de dato que permite definir un conjunto de valores predefinidos para una columna (Python 2024).

8.4. Entornos de Trabajo y Despliegue

Como se ha comentado anteriormente, para desarrollar esta aplicación se ha hecho uso de la herramienta GitLab, la cual facilita el control de versiones, la colaboración en el proyecto y el despliegue en diversos entornos. Se cuenta con los siguientes entornos:

- **Local (localhost:3000)**: Entorno local en el que los desarrolladores llevan a cabo las diferentes tareas a realizar.
- **Dev (<https://app.dev.revivack.red.upc.edu>)**: Entorno de desarrollo donde se integran las diferentes funcionalidades desarrolladas por diferentes colaboradores del proyecto y se realizan ciertas pruebas.
- **Pre (app-pre.revivack.com)**: Entorno de preproducción en el que se simula el entorno de producción para asegurarse de que todas las partes de la aplicación funcionan correctamente juntas antes de desplegar en producción.
- **Prod (app.revivack.com)**: Entorno de producción (entorno final) donde la aplicación es utilizada por los usuarios finales.

El proceso de despliegue¹⁶ comienza cuando, gracias al archivo llamado `gitlab-ci.yml`, se realiza un push o merge en una rama específica (pre, pro o master) en el repositorio. GitLab CI/CD (GitLab CI/CD es una herramienta de integración y entrega continua que automatiza la ejecución de pipelines para construir, probar y desplegar aplicaciones) detecta el cambio y asigna la variable de entorno ENV correspondiente (pre para pre, pro para pro y dev para master). Después, la pipeline ejecuta el trabajo `infrastructure` en la etapa `deploy`, donde Terraform (Terraform es una herramienta de infraestructura como código que permite definir y gestionar la infraestructura de manera declarativa (Hashicorp 2024)) inicializa y aplica la configuración necesaria para la infraestructura (dependiendo del valor de ENV, se realizan cosas distintas, por ejemplo, el valor de la `environment.name` o `environment.url` cambian). Una vez completado este trabajo, se ejecuta `deploy-compose`, que inicia sesión en Docker, prepara el entorno de despliegue copiando `compose.yml` y configurando las variables de entorno, y luego utiliza Docker Compose para descargar las imágenes y levantar los servicios definidos en `compose.yml`. Finalmente, se cierra la sesión en Docker, completando así el despliegue de la aplicación. A continuación se muestra un ejemplo de este archivo y como sigue ese proceso:

¹⁶La lógica detrás de los despliegues ha sido implementada por la Eseeleos y no por el autor este apartado es a modo de explicación.

```

include:
  - project: "eseleos/gitlab-ci"
    file:
      - "base.yml"
      - "build.yml"
      - "deploy.yml"
workflow:
  rules:
    - if: $CI_COMMIT_BRANCH == "master"
      variables:
        ENV: dev
build:
  extends: .build-docker
  script:
    - DOCKER_BUILDKIT=1 docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
      --build-arg SENTRY_AUTH_TOKEN=$SENTRY_AUTH_TOKEN
      --build-arg SENTRY_RELEASE=$CI_COMMIT_SHORT_SHA
      --build-arg NEXT_PUBLIC_SENTRY_DSN=$NEXT_PUBLIC_SENTRY_DSN
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
deploy:
  tags: ["dev", "shell"]
  before_script:
    - echo "$CI_REG_TOKEN" | docker login -u "$CI_REG_TOKEN_USERNAME" --password-stdin "$CI_REGISTRY"
  after_script:
    - docker logout $CI_REGISTRY
  script:
    - NEXT_DOCKER_TAG=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA docker compose -f compose.dev.yml up -d

```

Figura 45: Ejemplo del archivo `gitlab-ci.yml` para el despliegue en dev. Fuente: elaboración propia.

El código de la figura 45 define una pipeline para la construcción y despliegue de una aplicación Docker. A continuación se explican sus componentes principales:

- En el apartado de `include` Se incluyen configuraciones adicionales desde otro proyecto (tres archivos en este caso `base.yml`, `build.yml` y `deploy.yml`).
- En el apartado de `workflow` se asigna la variable `ENV` a `dev` cuando la rama es `master`.
- En el apartado de `build` se extiende una configuración base para construir una imagen Docker con argumentos específicos, y luego empuja la imagen al registro.
- En el apartado de `deploy`, se inicia sesión en Docker, despliega la aplicación usando Docker Compose con la configuración `compose.dev.yml`, y luego cierra la sesión en Docker.

9. Validación

9.1. Testing Unitario

Para testear las funcionalidades principales de la aplicación, Vitest ha sido utilizado como entorno de pruebas para la realización de tests unitarios de las funcionalidades principales desarrolladas. Vitest, una basada en Vite, ha demostrado ser especialmente adecuada para aplicaciones de JavaScript y TypeScript debido a su rendimiento optimizado y su integración con estas tecnologías (Vite 2024).

Este entorno de pruebas ofrece características avanzadas como la ejecución paralela de tests y una interfaz de línea de comandos, lo cual ha facilitado la identificación y corrección de errores en el código. También admite varias utilidades de test como `describe`, `it` y `expect`, que se utilizan comúnmente en el desarrollo dirigido por comportamiento (BDD)¹⁷.

A continuación se proporciona un ejemplo de un test unitario empleado:

```
import { describe, expect, it } from 'vitest';
import { isValidPlan } from './plan';

describe('isValidPlan', () => {
  it('should return true for a valid plan', () => {
    expect(isValidPlan('FREE')).toBe(true);
    expect(isValidPlan('BASIC')).toBe(true);
    expect(isValidPlan('PLUS')).toBe(true);
  });

  it('should return false for an invalid plan', () => {
    expect(isValidPlan('invalid')).toBe(false);
    expect(isValidPlan('trial')).toBe(false);
    expect(isValidPlan('enterprise')).toBe(false);
  });

  it('should be case-sensitive', () => {
    expect(isValidPlan('fREE')).toBe(false);
    expect(isValidPlan('bASIC')).toBe(false);
    expect(isValidPlan('PLUS')).toBe(false);
  });
});
```

Figura 46: Ejemplo de test unitario para el plan de una empresa. Fuente: elaboración propia.

En la figura 46 se puede observar como Vitest se utiliza para probar la funcionalidad de la función `isValidPlan`, que verifica si un nombre de plan dado es válido. Las pruebas están estructuradas para verificar tanto escenarios positivos como negativos, asegurando que la función

¹⁷Es una metodología Agile de desarrollo software en la que una aplicación se documenta y diseña en función del comportamiento que los usuarios esperan experimentar al interactuar con ella (Fuentes 2023).

identifique correctamente los nombres de planes válidos (FREE, BASIC y PLUS) y rechace los inválidos (invalid, trial y enterprise). Además, las pruebas confirman que la función es case sensitive (sensible a mayúsculas y minúsculas), lo cual es importante para mantener la integridad de los datos.

9.2. Testing Manual

Para asegurar la calidad y funcionalidad de las pantallas desarrolladas, se ha implementado un proceso de validación en los entornos de desarrollo y preproducción. Este proceso involucra tanto al equipo de desarrollo interno como a participantes externos al equipo. El objetivo principal de estos tests manuales es identificar y corregir cualquier fallo antes del despliegue en el entorno de producción.

Primero, las pantallas son sometidas a pruebas exhaustivas por los miembros del equipo en el entorno de desarrollo. Cada desarrollador realiza una serie de pruebas manuales simulando el uso real del sistema por parte de un usuario final. Durante estas pruebas, los desarrolladores se aseguran de que todas las funcionalidades operan correctamente y que la experiencia de usuario es fluida y sin errores.

Posteriormente, las pantallas se ponen a disposición de usuarios externos al equipo de desarrollo en el entorno de preproducción. Estos usuarios, que pueden incluir tanto empleados de otras áreas de la empresa como usuarios finales seleccionados, prueban las pantallas en condiciones que simulan el uso real de la aplicación. Se les pide que realicen tareas específicas y reporten cualquier problema o fallo que encuentren. Este feedback ofrece una perspectiva fresca y ayuda a identificar problemas que pueden haber pasado desapercibidos durante las pruebas internas en el entorno de desarrollo. Esta fase de validación externa asegura que las pantallas no solo funcionen técnicamente bien, sino que también cumplan con las expectativas y necesidades de los usuarios finales.